A Blockchain-Enabled Zero Trust Architecture for Industrial IoT Identity Management: Tiered Continuous Authorization and Quorum-Governed Revocation

Author Name[1], Author Name[2]
[1]Department of Computer Science, University Name, City, Country
[2]Department of Computer Science, University Name, City, Country
,

Abstract

Industrial plants combine long-lived devices, brownfield equipment that cannot hold cryptographic keys, several organizations with a stake in every asset, and requests whose consequences range from reading a temperature to rewriting a controller's program. Identity systems built on one certificate authority, session tokens or periodically refreshed revocation lists fit this setting poorly: a revoked device keeps working until a token expires or a list is refreshed, no single party is trusted by all to revoke it, and one trust threshold serves every request. This paper presents a blockchain-enabled Zero Trust architecture for Industrial IoT identity management. Three smart contracts on a permissioned proof-of-authority ledger hold device identities and status, a tiered access policy with an anomaly hard gate and a strict mode for safety-relevant writes, and a quorum-voting revocation governance. Zone gateways decide routine requests locally against replicated ledger state and anchor Merkle roots of their decision logs, while high-consequence requests are decided on-chain; batch revocation supports firmware recalls. We prove immediate revocation on the on-chain path, a bounded exposure on the fast path, replay resistance, attestation unforgeability, key-reuse prevention and governance safety, and characterize when a spoofing attacker is accepted. On an Ethereum Virtual Machine, registration costs about 215k gas, an on-chain decision about 70k, anchoring 54 gas per decision, and a quorum revocation 313k–800k gas for 5–21 voters; batching cuts recall gas by about 40%. On a synthetic fleet the on-chain path accepted no request after revocation, against 33–147 for token and list baselines. With the behavioural signal taken from real TON_IoT traffic, the full policy accepted under 1% of attack requests of seen types at the critical tiers but 6.6–11.9% of held-out types on average, and 29–77% when an adaptive attacker sent only its best-scoring flows; telemetry alone was a weak signal. Two further datasets from industrial testbeds showed that the result depends on the detector: a supervised detector failed on attack types withheld from training (75–78% of such requests accepted on average at the upper tiers of the third dataset), whereas a detector trained on normal traffic only blocked them at the cost of denying 12–22% of legitimate requests at the two critical tiers. On a four-validator QBFT network (Hyperledger Besu, 2 s blocks), audited calls were final in about one block period (2.0 s) both on one host and with the validators in four Google Cloud regions; across regions a revocation was visible on a remote validator 1.8 s after submission, about 0.2 s after the submitting node's receipt, and denied all 30 later audited requests, a three-of-five quorum revocation took 4.3–8.2 s, and sustained throughput was 31–50 decisions per second (11–18 on one host), still far below the gas-based ceiling; with two of four validators down the chain halted safely and resumed after recovery. These are single runs on a cloud provider's backbone without injected delay or partitions, and posture and network signals remain synthetic. Code is released at github.com/irfanrazanaqvi/BCZT_IDM_IoT_experments_code.

Index Terms—Industrial IoT, Zero Trust Architecture, Blockchain, Decentralized Identifiers, Identity Management, Smart Contracts, Revocation Governance, Operational Technology.

1 Introduction

1.1 Motivation

An industrial plant is a poor fit for the assumptions behind most identity systems. Its devices live for fifteen or twenty years, speak protocols that predate public-key cryptography, sit behind process-critical networks that cannot be rebooted at will, and are administered by several organizations at once: the operator who runs the line, the original manufacturer who ships firmware, an integrator who commissioned the cell, and sometimes an outside maintenance contractor. Guidance for operational technology (OT) security is explicit that availability and safety, not confidentiality, usually dominate the design space, and that a control that interrupts a process can be worse than the attack it prevents [1]. At the same time the attack surface has widened, because IT–OT convergence connects field assets to analytics, remote support and cloud services. Zero Trust, which drops trust based on network location and demands that every access be authenticated, authorized and re-evaluated [2, 3, 4], is a natural response, and early proposals adapt it to critical OT systems [5, 6].

Identity is where the difficulty concentrates. A conventional public-key infrastructure gives each device a certificate and relies on revocation lists or short certificate lifetimes to with-

draw it. In a multi-party plant, however, there is rarely one party that every other party is willing to let revoke a controller, and the delay between the decision to withdraw a device and the moment every gateway honours it is exactly the window in which a compromised or recalled device keeps working. Blockchain-based identity management addresses the first problem by anchoring identifiers, keys and status on a tamper-evident ledger that several organizations operate together, following the decentralized-identifier (DID) model [7, 8, 9, 10]. Recent industrial work builds authentication on this foundation [11], and surveys of blockchain for Industrial IoT (IIoT) catalogue its promise and its open problems [12]. Studies that combine blockchain with Zero Trust have followed [13, 14, 15, 16, 17, 18, 19].

1.2 Problem Statement and Gap

Reading this literature with an operator's eye, three things are missing. First, most blockchain-IoT identity schemes still authenticate once and then treat the session as trusted, which is precisely what Zero Trust is meant to remove [20, 21]; and the Zero Trust proposals for industrial settings generally assume one organization owns the policy decision point [22, 6]. Second, a single trust threshold is applied to every request, although reading a temperature and rewriting a safety-controller setpoint carry very different consequences; and, as we show analytically and experimentally, a fixed threshold on a weighted score can be unreachable by an attacker who spoofs most of the signals. Third, nearly all of these designs implicitly place every decision on the ledger, which cannot meet the rates and latency of plant networks, and none of them says how a manufacturer's firmware recall, a legacy device that cannot hold a key, or a lost connection to the ledger should be handled. We therefore ask:

How can a consortium-operated blockchain identity registry be combined with continuous, consequence-aware authorization so that (i) revocation, including large-scale firmware recalls, takes effect without a token or list-refresh window and without any single party being able to impose it, (ii) the strength of verification scales with the criticality of the resource, and (iii) the enforcement path stays fast, available and auditable on industrial networks?

1.3 Contributions

This paper makes the following contributions.

- **Architecture.** A zone-aware design (Section 4) with three smart contracts (identity registry, Zero Trust access policy, quorum revocation governance), a tiered policy model that ties the required trust to the criticality of the target resource, an anomaly hard gate and a strict mode for safety-relevant writes, a protocol-gateway identity proxy for legacy devices, and batch revocation and quarantine for firmware recalls.
- **Hybrid enforcement.** A two-path decision model: a fast path in which a zone gateway decides locally against replicated ledger state and anchors a Merkle root of its decision log, and an audited path in which high-consequence requests are decided by the contract itself (Section 4.5).
- **Analysis.** A threat model for plant environments and six proved properties, including a bound on how long a revoked device can still be accepted on the fast path and a result that exactly characterizes what the anomaly gate does and does not stop (Sections 3–5).
- **Open implementation and measurement.** Solidity 0.8.24 contracts executed on an EVM with measured gas for every operation, including recall and anchoring, plus a Python testbed using real Ed25519 signatures. Experiments report confidence intervals, a revocation-exposure comparison against token and CRL baselines, per-tier attack and false-denial rates, a gate sensitivity study, quorum-governance behaviour for 5–21 voters and recall capacity (Section 7). A further evaluation replaces the synthetic behavioural signal with detector scores on real TON_IoT traffic, including attack types withheld from training and an adaptive attacker (Section 7.7), repeated on a SCADA dataset (Section 7.8) and on an industrial dataset with labelled attack types (Section 7.9). The contracts are also deployed on a four-validator Hyperledger Besu QBFT network, first on one host and then with one validator in each of four Google Cloud regions, where we measure finality, revocation visibility, throughput, quorum voting, batch recall and validator failure (Section 7.14). We state plainly which results are compute-bound and which remain synthetic.

2 Related Work

2.1 Zero Trust for Industrial and OT Systems

Zero Trust architecture is by now well surveyed for IoT. Azad et al. cover continuous verification and micro-segmentation [23], James et al. review authentication and authorization schemes specifically under a Zero Trust model [21], and Nahar et al. examine the 6G setting, where device density sharpens per-request overhead concerns [24]. Fernández and Brazhuk caution that "Zero Trust" is sometimes a label without a falsifiable architecture behind it [3], and Phiayura and Teerakanok supply a migration framework with a maturity model [4]. For OT in particular, Song et al. propose a Zero Trust architecture for critical operational technology systems [5], Zanasi et al. a flexible architecture for IIoT [6], and Kumar and Sharma integrate blockchain with dynamic access control for Industry 4.0 without a DID-anchored registry [22]. These works establish the principles; they leave open who owns revocation in a multi-party plant and how a decision should be audited afterwards.

2.2 Blockchain Identity and Authentication for IIoT

Surveys of blockchain-based decentralized identification note limits around revocation latency and interoperability [8], and a broad review of blockchain in IIoT discusses security and privacy opportunities alongside scalability and integration costs [12]. Aguru and Erukala propose a blockchain self-sovereign-identity framework for standardized IoT identification [9], Folino et al. tie identity to a physically unclonable function as a hardware root of trust [10], and Ren et al. combine self-sovereign identity with Zero Trust for AIoT using selective-disclosure credentials [25]. On the industrial authentication side, Zhai et al. design a lightweight scheme for IIoT that combines blockchain with Chebyshev chaotic maps to suit constrained devices [11]. DAXiot shows that DID challenge–

response for MQTT can run without a blockchain at about 21% connection overhead over mutual TLS, but then has no shared revocation ledger [26]. For revocation itself, Mazzocca et al. propose EVOKE, a cryptographic-accumulator mechanism with about 1.5 KB of per-device state that spreads revocation epidemically among devices [27]; it complements an on-chain status check rather than replacing it. Kumar et al. run a privacy-preserving identity system on a private Ethereum proof-of-authority (PoA) chain in which a decentralized voter group governs identity updates and revocations, and report roughly 3–4 s for registration and under 5 s for revocation on the live network [28]; it is the closest published precedent for our governance contract and the only live-chain figure we compare against.

2.3 Blockchain–Zero Trust Combinations

Nie et al. pair smart-contract policy enforcement with inner-product encryption for fine-grained access in 6G IoT [14]. Salim et al. propose ZT-BlocIoT with dynamic trusted gateways and reinforcement-learning-driven sharding, reporting 2,500 transactions per second on 700 nodes at 42 ms latency [15]. Wang et al. observe that even Zero Trust–inspired schemes for medical IoT typically authenticate once and then trust the session, and propose blockchain-backed continuous authentication [20]; the same gap is what motivates our per-request design in industry. Mukta et al. delegate Zero Trust access control through a blockchain, which is close to our policy layer but concerns delegation rather than the device identity lifecycle [16]. For smart factories, Farhan et al. describe a conceptual Zero Trust framework on a consortium chain [18], and Potturi et al. combine AI risk scoring with blockchain-anchored federated identity across cloud, edge and IoT [19]. Jamwal et al. conclude that blockchain-enabled Zero Trust is still mostly proposed at survey level rather than delivered as benchmarked open code [17], and a bibliometric analysis finds blockchain with Zero Trust identity management to be one of the field's most active clusters [13]. Usama et al. add decentralized identity, zero-knowledge proofs and Byzantine-resilient federated learning against AI-driven identity attacks [29]. Federated learning is also studied as a way to share threat intelligence across industrial sites without exposing raw data [30]; we do not use it, but it is a plausible source for the anomaly signal in Section 4.4.

2.4 Off-Chain Dependencies, Cost and Oracles

Because on-chain controls rest on off-chain facts, the reliability of the data that reaches a contract matters. Shujaa et al. stress that device attestation, secure boot and key custody remain off-chain dependencies of any blockchain-IoT security design [31], and Hassan et al. survey the oracle problem, that is, how to bring trustworthy external data to a contract [32]; our trust oracle is an instance of it, and Section 8 discusses its single-key weakness. Smart-contract defects are another risk, reviewed by Wei et al. [33]. On cost, gas has been studied for Ethereum smart contracts [34] and for IoT access control [35], DPoS and PBFT-based consensus choices have been proposed for IoT and clinical settings [36, 37], and gasless relaying can spare constrained devices from paying fees [38]. Ren et al. show for vehicular networks that published trust values can themselves link pseudonyms and propose differential privacy to prevent it [39], which matters for any design that exposes scores on a shared ledger.

2.5 Positioning

Table 1 compares the most relevant systems on five properties that matter in a plant: on-chain revocation, continuous per-request authorization, consequence-aware (tiered) policy, handling of legacy and recall scenarios, and a released artifact. The assessments rest on the abstracts and the parts of the full texts we could read; "not stated" means we found no statement, not that the property is absent. To our knowledge none of the reviewed systems reports all five.

3 Industrial Requirements, System and Threat Model

3.1 Requirements Specific to Industrial Settings

Translating the generic Zero Trust and blockchain-identity goals into a plant yields six requirements that drive the design; they are our reading of OT security guidance [1] and of the identity literature surveyed above, not a standard.

- **R1 Multi-party authority.** No single organization (operator, manufacturer, integrator) should be able to unilaterally revoke or reinstate a device.
- **R2 Consequence-aware verification.** The evidence required before a request is accepted must grow with the consequence of the target resource: reading telemetry is not rewriting a controller's program.
- **R3 Predictable, available enforcement.** The common-case decision must be fast and must not depend on the ledger being reachable; consensus latency is acceptable only for rare, high-consequence actions.
- **R4 Brownfield coexistence.** Devices that cannot hold a key or run a signature scheme must be representable without weakening the identities of those that can.
- **R5 Recall at fleet scale.** Withdrawing a vulnerable firmware generation must be possible for thousands of devices in a bounded number of blocks.
- **R6 Auditability.** Every decision, including those taken locally for speed, must be attributable and tamper-evident after the fact.

Scope. We target the supervisory, engineering and maintenance plane: historian writes, setpoint changes, program and firmware downloads, and telemetry reads. Hard real-time cyclic I/O between a controller and its field devices, with millisecond or microsecond determinism, is outside the scope: per-message authorization there would be inappropriate, and such traffic remains protected by segmentation and by device-level mechanisms that this architecture only complements.

3.2 System Model and Assumptions

Devices $\mathcal{D}$ hold signing keys (sk_i, pk_i) generated at commissioning, except for a *legacy* subset that is represented by a protocol gateway holding a proxy identity. Zone gateways act as Policy Enforcement Points (PEPs). A trust oracle $\mathcal{O}$ with key

Table 1: Comparison of representative blockchain and Zero Trust systems for (I)IoT identity

System	Core mechanism	On-chain revocation	Per-request authZ	Tiered policy	Legacy / recall	Open artifact
Zhai et al. 2025 [11]	Blockchain + Chebyshev-map authentication (IIoT)	Not stated	Not stated	Not stated	Lightweight devices	Not stated
Song et al. 2025 [5]	ZTA for critical OT	Not stated	Not stated	Not stated	Not stated	Not stated
Zanasi et al. 2024 [6]	Flexible IIoT ZTA	No	Yes	Partial	Not stated	Not stated
Kumar & Sharma 2024 [22]	Blockchain + dynamic access control (Industry 4.0)	Not stated	Yes	Not stated	Not stated	Not stated
Farhan et al. 2026 [18]	ZT on consortium chain (smart factory)	Conceptual	Conceptual	Not stated	Not stated	Not stated
Potturi et al. 2026 [19]	AI risk score + chain-anchored identity	Yes	Yes	Not stated	Not stated	Not stated
Salim et al. 2026 [15]	ZT-BlocIoT (trust + DRL sharding)	Yes	Yes	Not stated	Not stated	Not stated
Nie et al. 2025 [14]	ZT + inner-product encryption (6G)	Yes	Yes	Fine-grained attributes	Not stated	Not stated
Wang et al. 2026 [20]	Blockchain continuous auth. (medical IoT)	Yes	Yes	No	Not stated	Not stated
Kumar et al. 2026 [28]	Private PoA identity + voter group	Yes (voter group)	Not stated	No	Not stated	Not stated
This work	**DID registry, tiered ZT contracts, quorum governance, hybrid enforcement**	**Yes (quorum-governed, batch)**	**Yes (every call)**	**Yes (4 tiers, gate, strict)**	**Proxy identity; batch recall; outage mode**	**Yes (Solidity + Python)**

$sk_{\mathcal{O}}$ attests the context signals. A registrar role is held by a provisioning service, and an optional set of N voters (for example the manufacturer, the operator and an auditor) governs revocation. A consortium of V validators runs a permissioned PoA chain. We assume:

- **A1 (ledger integrity).** Fewer than half of the validators are Byzantine, so finalized state is immutable and visible to subsequent contract calls.
- **A2 (cryptography).** Ed25519 and ECDSA over secp256k1 are existentially unforgeable under chosen-message attack (EUF-CMA); SHA-256 and Keccak-256 are collision resistant.
- **A3 (key custody).** A device key stays secret until the adversary compromises that device. Attestation, secure boot and key custody are off-chain dependencies the ledger cannot enforce [31].
- **A4 (role keys).** The policy-administrator and oracle keys are uncompromised, and at most $q-1$ voter keys are compromised.
- **A5 (replica freshness).** A zone replica applies each finalized registry event within Δ seconds of finality. Δ is an operational parameter (for example the block interval plus a polling or subscription delay), not a protocol guarantee.

3.3 Adversary Model and Security Goals

The adversary controls the network between devices and PEPs (observe, replay, drop, inject), may compromise a bounded set of devices and obtain their keys, may enrol devices of its own, and controls the context signals reported by a device it has compromised. It cannot break A1–A5. In addition, we consider an *insider*: a single organization that holds the registrar role and tries to revoke or reinstate devices for its own advantage. Table 2 maps threats to mitigations and evidence.

The goals are: **G1** no request is accepted from a revoked, suspended or unknown identity, immediately on the audited path and within Δ on the fast path; **G2** each request is accepted at most once; **G3** a device cannot raise its own trust or lower its own anomaly value; **G4** a compromised key cannot be reused under a new identity; **G5** the effect of any single context signal, and the conditions under which a spoofing attacker is accepted, are known exactly; **G6** fewer than q voters cannot revoke a device, and fewer than $N-q+1$ cannot block a revocation that q voters approve; and **G7** every decision, including fast-path ones, is covered by an anchored commitment.

Table 2: Threat model: adversary actions, mitigations and evidence

Threat	Mitigation	Evidence
T1 Replay	Nonce set (audited) or monotonic counter (fast path)	Prop. 3; Sec. 7.12
T2 Impersonation / forgery	Signature checked against on-chain key	Sec. 7.12
T3 Use after revocation	Status first on every call; terminal REVOKED	Props. 1, 2; Sec. 7.10
T4 Re-enrolment of a revoked key	Key-hash blacklist	Prop. 5
T5 Self-declared trust or tampered anomaly value	Oracle signature over $(\mathrm{did}, \mathrm{res}, T, A, \mathrm{nonce})$	Prop. 4
T6 Signal spoofing / mimicry	Bounded signal weights, anomaly gate, strict tiers	Prop. 7; Secs. 7.5, 7.6
T7 Insider revokes or reinstates at will	Quorum governance	Prop. 6; Sec. 7.13
T8 Legacy device impersonated behind a gateway	Proxy identity limited to low tiers	Sec. 4.3
T9 Ledger or replica outage	Tier-dependent degraded mode	Sec. 4.8
T10 Validator collusion, oracle compromise	Out of scope (A1, A4)	Sec. 8

4 Proposed Architecture

4.1 Overview

Figure 1 shows the design. Devices and a protocol gateway sit in the field zone; the zone PEP enforces decisions; the trust engine (policy decision point, PDP), trust oracle and a local replica of the chain state sit in the site operations zone; and the three contracts are operated by the consortium. The PEP/PDP split follows the classical Zero Trust reference model [2, 3], but the policy store and the identity registry are held by the consortium rather than by one site, so that no gateway can edit another stakeholder's view of a device.

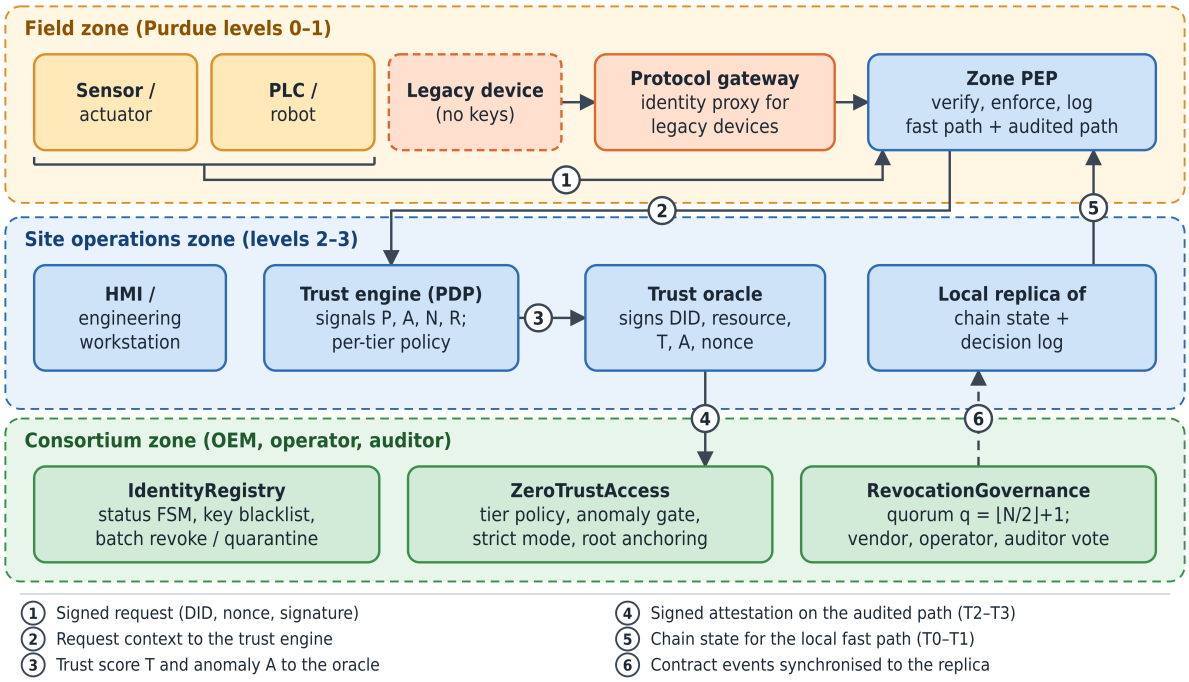


Figure 1: Architecture of the blockchain-enabled Zero Trust system for Industrial IoT identity management, organized by plant zone. Text description: three stacked zones; in the field zone, sensors, PLCs and legacy devices reach a protocol gateway and a zone policy enforcement point (fast path plus audited path); in the site operations zone, a trust engine and trust oracle feed the audited path and a local replica of the chain serves the fast path; the consortium zone holds the three contracts (IdentityRegistry, ZeroTrustAccess, RevocationGovernance), whose events synchronise the replica. Numbered arrows: (1) signed request, (2) request context to the trust engine, (3) trust score and anomaly signal to the oracle, (4) signed attestation on the audited path (T2–T3), (5) chain state for the local fast path (T0–T1), (6) contract events to the replica.

4.2 Identity Lifecycle and the Registry

A device derives a DID from its public key in the style of the W3C data model [7], $\mathrm{did} = \texttt{did:iot:}\,\mathrm{H}(pk)[0{:}32]$, with SHA-256 in the testbed and Keccak-256 in the Solidity track. Commissioning submits pk (or its hash), a device class, and a hash of the declared attributes (firmware version, vendor, location), signed by sk so that the record proves key possession without exposing the attributes. The registry is a state machine:

$$\text{UNKNOWN} \xrightarrow{\text{register}} \text{ACTIVE}$$

$$\text{ACTIVE} \xrightarrow{\text{suspend}} \text{SUSPENDED} \xrightarrow{\text{reinstate}} \text{ACTIVE}$$

$$\text{ACTIVE or SUSPENDED} \xrightarrow{\text{revoke}} \text{REVOKED (terminal)}$$

REVOKED is terminal, and revocation also blacklists the key hash, so the same key cannot be enrolled under a new DID. Suspension is the reversible quarantine used while an advisory is assessed. All transitions are restricted to the registrar role and emit indexed events, which zone replicas and security monitoring consume.

4.3 Legacy Devices Behind a Protocol Gateway

Brownfield equipment often cannot sign requests. We represent each such device by a DID derived from the *gateway's* key and the device's local address, registered with the class `legacy`. The gateway signs on its behalf and supplies weaker posture evidence (the testbed draws legacy posture from a lower range than native devices). Two rules keep this from lowering the security of the whole plant: the class check admits `legacy` identities only to low-consequence resources (tiers T0–T1 in our policy), and revoking the gateway key revokes every proxied identity at once. The trade-off is explicit: a compromised gateway can speak for every legacy device behind it, which is why this class never reaches the strict tiers.

Table 3: Criticality tiers used in the evaluation

Tier	Example resource	θ	m	g	Classes
T0	telemetry.read	0.50	0.10	1.00	all, incl. legacy
T1	historian.write	0.55	0.10	0.60	all but HMI
T2	plc.setpoint.write†	0.70	0.05	0.30	HMI, PLC, robot
T3	plc.firmware.update†	0.78	0.04	0.20	PLC, robot

† strict: borderline scores are denied rather than stepped up.

4.4 Tiered Zero Trust Policy

Each resource has a policy $\pi = (\theta, m, g, s, \mathcal{C}, \tau_{\max})$: a minimum trust θ, a step-up margin m, an anomaly hard gate g, a strict flag s, a set of permitted device classes $\mathcal{C}$ and a maximum idle time $\tau_{\max}$. Table 3 lists the four tiers used in our evaluation. The parameters are illustrative and tuned by hand to our synthetic signal ranges; a real plant would set them from its own telemetry and risk assessment.

The trust score combines four signals in $[0, 1]$: device posture P (attested firmware and configuration state), behavioural anomaly A, network context N and historical reliability R (mean of the last ten scores, 0.6 for a new device),

$$T = w_p P + w_b(1 - A) + w_n N + w_r R, \qquad (1)$$

with default weights $(w_p, w_b, w_n, w_r) =$

$(0.30, 0.30, 0.20, 0.20)$, The signals come from the PEP and from monitoring that the plant already operates; in practice A could be produced by process-aware anomaly detection, possibly trained across sites without sharing raw data [30]. We treat their production as an input and evaluate only how the policy handles them. The oracle signs $(\text{did}, \text{res}, T, A, \text{nonce})$; including A in the signed message is what lets a contract enforce the gate without trusting the device's own report.

Algorithm 1 `evaluateAccess(did, res, class, T, A, nonce, sig)`

1. **if** status(did) $\neq$ ACTIVE **or** no policy for res **then return** DENY
2. **if** (did, nonce) already used **then return** DENY; mark it used
3. **if** class $\notin \mathcal{C}$ **then return** DENY
4. **if** oracle signature over (did, res, T, A, nonce) is invalid **then return** DENY
5. **if** $A > g$ **then return** DENY *(anomaly gate)*
6. **if** $T < \theta$ **then return** DENY
7. **if** $T < \theta + m$ **then return** (**if** strict **then** DENY **else** STEP_UP)
8. **return** ALLOW

The gate and the strict flag address a specific weakness of weighted scores that Proposition 7 makes precise: because the score is linear, an attacker who spoofs posture, network and reliability can hold T high no matter how abnormal its behaviour is, so for consequential resources the anomaly signal must be able to veto independently of the sum. For a step-up, we leave the second-factor mechanism to the plant (for example a local operator confirmation); STEP_UP is never returned for strict resources.

4.5 Hybrid Enforcement: Fast Path and Audited Path

Running Algorithm 1 on the chain for every request would cap throughput at what a block can carry and would couple plant operations to consensus latency. We therefore split enforcement by tier.

- **Fast path (T0, T1).** The zone PEP decides locally against its replica: status and key from the registry events, a per-device monotonic counter in place of the nonce set, the device's request signature, the oracle's attestation and the tier policy. Each decision appends a leaf to a log; every B decisions the PEP computes a Merkle root and anchors it with `anchorDecisions(root, count)`, which only authorized PEP addresses may call. An auditor can later check any single logged decision against the anchored root.
- **Audited path (T2, T3).** The request is decided by `evaluateAccess` on the chain, so the decision, its inputs and its outcome are themselves on the ledger. These resources are written rarely and under operator control, which is what makes the cost acceptable; if a plant's setpoint writes are frequent, this assumption fails and the tiers must be recut.

The price of the fast path is the replica lag Δ of assumption A5: a device revoked just before a request may still be accepted for tier T0–T1 resources until the replica catches up (Proposition 2). The audited path has no such window.

4.6 Firmware Recall and Quarantine

When a vulnerability is disclosed in a firmware generation, the manufacturer needs to withdraw every affected device quickly. `revokeBatch(dids, reason)` revokes many devices in one transaction, skips unknown or already-revoked entries rather than reverting (so one stale entry cannot block a recall), blacklists each key and emits one event per device; `suspendBatch` quarantines them reversibly. The batch is limited by the block gas limit, and Section 7.11 measures its per-device gas and the number of blocks needed for fleets of 10^3–5×10^4 devices.

4.7 Decentralized Revocation Governance

A single registrar can deny service to legitimate devices, or reinstate a dangerous one. `RevocationGovernance` mediates `revokeDevice()` through threshold voting among N independent voters, in the spirit of the voter group of [28]. A voter calls `propose(did, reason)`; each voter casts one `castVote(id, approve)`. The proposal executes the revocation as soon as approvals reach $q = \lfloor N/2 \rfloor + 1$, and is finalized as rejected once denials exceed $N - q$. The governance contract holds the registrar role on the registry; time-critical actions that a signed vendor bulletin pre-authorizes may use the direct path, so the two paths coexist. Votes are non-transferable, one per voter, and DENY votes never count toward approval.

4.8 Degraded Operation During an Outage

If a PEP loses its replica feed, it must pick between availability and safety. Our policy is tier-dependent: T0–T1 requests continue against the last known state (accepting that a device revoked during the outage is not yet known), while audited-path requests (T2–T3) are *denied*, because their decision needs the chain. This is fail-closed for consequential writes and fail-operational for reads; whether an operator accepts that trade is a plant decision, and Section 7.17 quantifies it for a simple scenario.

5 Security Analysis

The contracts are deterministic state machines, so the proofs are short; they establish that the logic meets goals G1–G7 under A1–A5, not that a deployment is secure.

5.1 Security Properties

Proposition 1 (Immediate revocation on the audited path, G1)**.** *If `revokeDevice(did)` (or `revokeBatch` containing `did`) is in a finalized block, every later call to `evaluateAccess` for `did` returns DENY.*

Proof. By A1 the call sees finalized state with $\text{status}(\text{did}) =$ REVOKED. No function leaves REVOKED, and step 1 of Algorithm 1 runs before every other check and denies all non-ACTIVE statuses. □

Proposition 2 (Bounded exposure on the fast path, G1)**.** *Let a revocation finalize at time t_f and let a PEP satisfy A5. A fast-path request evaluated at time $\tau \geq t_f + \Delta$ is denied. If the device submits at most ρ accepted requests per second, at most $\rho\Delta$ requests are accepted after t_f.*

Proof. By A5 the replica shows status REVOKED from $t_f + \Delta$ on, and the fast path denies non-ACTIVE devices before any other check. Requests accepted after t_f therefore fall in $[t_f, t_f + \Delta)$, which holds at most $\rho\Delta$ of them. □

Proposition 3 (Replay resistance, G2)**.** *For a single PEP and any* (did, c) *with* c *a nonce (audited) or counter value (fast path), at most one evaluation returns ALLOW or STEP_UP.*

Proof. Audited path: the test-and-set of the used-nonce flag is one transaction and transactions execute serially, so a second evaluation sees the flag and denies at step 2. Fast path: the PEP accepts only counters greater than the stored maximum and then stores the new value, so a repeated or older counter is denied. In a multi-PEP zone, the PEP identity must be bound into the signed message so that a request captured at one PEP is useless at another; our testbed exercises a single PEP. □

Proposition 4 (Attestation unforgeability, G3)**.** *A PPT adversary without* $sk_{\mathcal{O}}$ *obtains ALLOW or STEP_UP for a pair* (T, A) *not attested by* $\mathcal{O}$ *for that request only with negligible probability.*

Proof. Steps 5–8 are reached only after the signature over $(\mathrm{did}, \mathrm{res}, T, A, \mathrm{nonce})$ verifies under $pk_{\mathcal{O}}$. Producing a different (T, A) with a valid signature is an EUF-CMA forgery (A2). The nonce is inside the message, so an attestation cannot be moved to another request. Changing only A (for example, reporting a lower anomaly) changes the signed message, which the contract tests demonstrate. □

Proposition 5 (Key-reuse prevention, G4)**.** *After a finalized revocation,* `registerDevice` *with the same key hash reverts.*

Proof. Revocation sets $\mathrm{revokedKey}[\mathrm{H}(pk)]$, and registration reverts on that flag. Since the DID is a function of $\mathrm{H}(pk)$, a new identity needs a new key, which the adversary lacks unless it compromised another device. □

Proposition 6 (Governance safety and liveness, G6)**.** *With* N *voters and quorum* q*: (a) a revocation executes only if at least* q *distinct voters approved; (b) a set of at most* $N - q$ *voters cannot stop a proposal that the other* q *approve; (c) a coalition of at most* $q - 1$ *voters cannot execute a revocation.*

Proof. (a) `hasVoted` blocks repeats and `isVoter` blocks outsiders; the approval counter reaches q only with q distinct approvals, and only then is `revokeDevice` called. (b) Denials end a proposal only when they exceed $N - q$, which at most $N - q$ dissenters cannot, so the q approvers can always complete. (c) Fewer than q approvals never trigger execution. Safety additionally requires that no other registrar path exists for the affected devices. □

Proposition 7 (Spoofing, the gate and strictness, G5)**.** *(a) If scores* T, T' *differ only in signal* s_j *with weight* w_j*, then* $|T - T'| \le w_j$*. (b) On a resource with gate* g*, every ALLOW satisfies* $A \le g$*. (c) On a strict resource, every ALLOW satisfies* $T \ge \theta + m$*. (d) For an attacker with* $P = N = R = 1$*,* $T = 1 - w_b A$*, and it is accepted on a strict resource if and only if* $A \le \min\{g, (1 - \theta - m)/w_b\}$*.*

Proof. (a) $T - T' = w_j(s_j - s'_j)$ and $|s_j - s'_j| \le 1$. (b) and (c) are steps 5 and 7 of Algorithm 1. (d) Substituting gives $T = 0.3 + 0.3(1 - A) + 0.2 + 0.2 = 1 - 0.3A$, and ALLOW needs $A \le g$ and $T \ge \theta + m$. □

Corollary (what a fixed threshold cannot do). With the default weights a fully spoofing attacker has $T \ge 0.70$ even at $A = 1$. Hence on a non-strict resource with $\theta = 0.55$ and no gate, *no* anomaly value gets such an attacker denied, and for the tiers of Table 3 the bound $(1-\theta-m)/w_b$ is 0.83 (T2) and 0.60 (T3), above the gates 0.30 and 0.20; the *gate*, not the score, is what stops the attacker, and the residual risk is exactly the set of attackers who keep $A \le g$. The experiments of Section 7.6 measure how that residual behaves.

5.2 Residual Risks

The properties say nothing about (i) compromised validators or oracle keys (A1, A4); (ii) a stolen key used by an attacker who behaves like the owner until revoked, which no continuous scheme separates from the owner except by behaviour; (iii) side-channel extraction of keys; or (iv) smart-contract defects, for which the quality-assurance literature recommends testing, verification and audit [33]. Also, the trust oracle is a trusted data source, which is the blockchain-oracle problem in a specific form [32]; a threshold-signed oracle would reduce it (Section 8). Finally, the nonce set costs one storage slot per accepted audited request and needs epoch-based pruning in a long-lived deployment.

6 Prototype Implementation

6.1 Two Tracks

Track A – Solidity on an EVM. `IdentityRegistry.sol`, `ZeroTrustAccess.sol` and `RevocationGovernance.sol` (Solidity 0.8.24, optimizer 200 runs) implement Section 4: the status machine, Algorithm 1 with anomaly gate and strict mode, `revokeBatch`/`suspendBatch`, PEP-authorized root anchoring, and quorum voting. They are compiled with solc-js and executed on the Hardhat in-process EVM. A Mocha/Chai suite of 16 tests passes (13 functional tests and three hardening and randomised-invariant tests, Section 7.16); besides the registry, replay, revocation and governance cases it covers the industrial additions: the gate denies a high composite score when $A > g$, a tampered anomaly value invalidates the oracle signature, strict resources never step up, batch revocation skips unknown entries and blocks later access, and only authorized PEPs can anchor. Scripts measure gas for every operation. This track gives real gas but, on its own, not multi-node consensus or network latency (Track C adds those), and the device's request signature is verified by the PEP rather than on-chain.

Track B – Python testbed. A Python implementation of the same state machine uses real Ed25519 (the `cryptography` library), SHA-256 and a Merkle root. Besides the contract model it contains the `FastPathPEP`, which verifies the device signature and the oracle attestation (two Ed25519 verifications), applies the per-device counter and tier policy, and logs decisions in Merkle-rooted batches. It is used for scale, statistical, baseline, tier and governance experiments. The host is a

2-vCPU Linux container (Python 3.13, Hardhat 2.22); all seeds derive from 20261004. Track B uses Ed25519 whereas Track A uses secp256k1 ECDSA, so their timings are not comparable.

Track C – Four-validator QBFT network. The same contracts are deployed on Hyperledger Besu 25.7.0 with four validators (QBFT, 2 s block period, 4 s round timeout, chain ID 2026, zero base fee, one JVM of 700 MB per node, ports 8545–8548) started by `start_net.sh` from `qbftConfigFile.json`. `besu_benchmark.js` (ethers v6) measures block interval, sequential call latency, revocation visibility on a second validator, throughput with pre-signed transactions from ten senders, quorum voting and batch recall, and `besu_fault.js` stops and restarts validators. In the first deployment everything, including the load generator, shares one two-vCPU host, which limits the throughput results (Section 7.14). In the second deployment each validator runs on its own e2-standard-2 virtual machine (2 vCPU, 8 GB) in europe-west3, us-east4, asia-southeast1 and southamerica-east1, with the load generator in europe-west3; the same scripts are used with the endpoint set by environment variable, and the provisioning scripts are part of the repository (Section 7.15).

6.2 Industrial Fleet and Signal Model

The synthetic fleet has six classes with proportions sensor 40%, PLC 15%, HMI 8%, robot 7%, gateway 15% and legacy 15%. Native devices draw posture in $[0.75, 1]$, anomaly in $[0, 0.15]$ and network context in $[0.7, 1]$; legacy devices draw posture in $[0.55, 0.75]$, anomaly in $[0, 0.20]$ and network in $[0.6, 0.9]$. To mimic process upsets, a fraction of legitimate requests (5% by default; 0, 5 and 15% in the sensitivity study) draws anomaly from $[0.15, 0.45]$. Four attack populations are used: *naive compromise* (all signals degraded), *partial mimicry* (posture and network legitimate, $A \in [0.3, 0.7]$), *stealthy mimicry* ($A \in [0.15, 0.40]$) and *full spoof* ($P = N = 1$, $A \in [0.3, 0.7]$); each starts from three clean prior requests. The legacy gateway of Section 4.3 is modelled through the class `legacy` and its weaker signals; deriving proxy DIDs from the gateway key is a design element we did not implement. In the synthetic experiments the distributions are our assumptions, not field measurements, and the attack and legitimate ranges overlap by construction, so they show how the policy responds to such overlap rather than how well a detector would perform in a plant. The real-data experiment of Section 7.7 (`sim/experiments_realdata.py`, scikit-learn 1.9) replaces only the behavioural signal.

6.3 Experiment Scenarios and Reproducibility

Carried over from the base design: (S1) scaling over $n \in \{50, \ldots, 1600\}$; (S2) 500 trials per basic attack; (S3) 30 repeated runs at $n = 400$; (S4) baseline revocation exposure for 60 devices at 1 request/s over 600 logical seconds with revocation times from $U(100, 500)$ s; (S5) governance with 200 trials per $N \in \{5, \ldots, 21\}$ under cooperation, maximal dissent and forced revocation; and a trust-signal ablation. *Industrial additions:* (X1) tiered policy on the mixed fleet, 4,000 requests per population; (X2) anomaly-gate sweep; (X3) fast path versus modelled on-chain path, 10 runs of 2,000 requests over 300 PLC and robot devices; (X4) recall capacity from measured gas; and (X5) outage scenario; and (R) the TON_IoT study of Section 7.7, with five seeds for detector quality and one seed for held-out types and the policy runs. The commands are `python -m sim.benchmark`, `sim.experiments_extra`, `sim.experiments_governance`, `sim.experiments_iiot`, `sim.experiments_realdata` (needs the TON_IoT CSVs in `data/`; they are not redistributed) and `hardhat run scripts/measure_gas.js`, `measure_governance_gas.js`, `measure_iiot_gas.js`; `paper/gen_tables.py` regenerates every table from the resulting JSON files.

Table 4: Scaling results, one run per fleet size (latency in ms, throughput in ops/s)

n	Reg. lat.	AuthZ lat.	AuthZ p95	Reg. tput	AuthZ tput
50	0.286	0.184	0.216	3,477	3,812
100	0.282	0.197	0.236	3,539	3,679
200	0.256	0.125	0.148	3,906	5,909
400	0.173	0.120	0.143	5,755	6,159
800	0.177	0.127	0.162	5,645	5,804
1,600	0.175	0.126	0.153	5,693	5,879

7 Experimental Evaluation

7.1 Methodology and What the Numbers Mean

We report decision latency and throughput, on-chain gas, revocation exposure (requests accepted after the revocation instant), per-tier acceptance of legitimate and attacking requests, quorum behaviour, recall capacity and a degraded-mode scenario. Latencies are wall-clock times from a monotonic timer. Three limits apply throughout. (i) Testbed numbers are *compute-bound*: they exclude network, block time and multi-node consensus. (ii) Sections 7.2–7.17 use *synthetic* devices, signals and attacks, so their detection-type results characterize policy mechanics under stated overlaps; Section 7.7 replaces the behavioural signal with detector scores computed on real TON_IoT traffic, while posture and network context remain synthetic. (iii) One two-vCPU host was used.

7.2 Testbed Performance and Repeated-Run Statistics

Table 4 gives one run per fleet size. Mean authorization latency is 0.120–0.197 ms and registration 0.173–0.286 ms; the two smallest fleets are slower (0.184 and 0.197 ms against 0.120–0.127 ms for $n \geq 200$), probably from warm-up, which we did not isolate. Beyond $n = 200$ latency is flat as the fleet grows eightfold, consistent with constant per-request cost dominated by one Ed25519 verification. Throughput is 3,700–6,200 decisions/s. Repeating $n = 400$ thirty times gives a mean authorization latency of 0.133 ± 0.013 ms (95% CI 0.128–0.138) and registration 0.184 ± 0.019 ms; over 20,000 sequential decisions the percentiles are p50 0.124, p95 0.188, p99 0.247 and p99.9 0.932 ms, with a 4.3 ms maximum attributable to scheduling or garbage collection. Intervals assume roughly normal run means and apply to this host.

Table 5: Measured gas per operation (Hardhat EVM, solc 0.8.24, 50 devices)

Operation	Mean gas	Std.
`registerDevice`	214,950	8.9
`suspendDevice`	32,709	5.9
`reinstateDevice`	32,901	5.9
`revokeDevice`	58,824	5.9
`evaluateAccess` (ALLOW)	70,368	9.7
`evaluateAccess` (STEP_UP)	70,413	8.0
`evaluateAccess` (DENY, bad oracle sig.)	70,143	8.3
`evaluateAccess` (DENY, replay)	41,526	9.7
`evaluateAccess` (DENY, revoked)	39,308	8.4
Deploy `IdentityRegistry`	1,333,022	–
Deploy `ZeroTrustAccess`	1,079,680	–
`definePolicy` (2 classes)	97,805	–

7.3 On-Chain Cost: Measured EVM Gas

Table 5 lists gas measured over 50 devices on the Hardhat EVM; gas is deterministic given the state path (s.d. below 10). Registration costs about 215k gas and a full audited decision about 70k, of which one `ecrecover` and one new nonce slot are the main parts. Requests denied for replay (41.5k) or against a revoked device (39.3k) are cheaper because they exit early, while denials that occur after the oracle check cost as much as an allow. The industrial additions cost nothing visible: an ALLOW on a strict resource (70.4k), an anomaly-gate denial (70.2k), a strict-borderline denial (70.4k) and a below-threshold denial (70.2k) are all within 0.3% of the base decision, since the extra checks are cheap comparisons. Anchoring a decision-log root costs 70.8k gas for the first anchor and 53.7k in steady state, which is 54 gas per decision at $B = 1000$ decisions per root, about three orders of magnitude below the 70.4k of deciding each of them on-chain.

For capacity planning, a 30M-gas block (a typical default; PoA operators choose their own) holds at most $\lfloor 30\text{M}/70{,}369 \rfloor = 426$ audited decisions or 139 registrations. This is a gas-based upper bound, not a measured throughput (Section 7.14 measures a much lower sustained rate on a real network), and it is the reason why only T2–T3 are decided on-chain. The in-process call latency including client-side signing was about 3 ms per decision, an upper bound on pure EVM execution that says nothing about network or consensus delay.

7.4 Hybrid Enforcement: Cost of the Fast Path

We compare the contract model, which verifies one signature per decision (Track B), with the fast-path PEP, which verifies the device signature and the oracle attestation. Over 10 runs of 2,000 mixed T0/T2 requests, the contract model takes 0.134 ± 0.020 ms (95% CI 0.119–0.148) and the fast path 0.261 ± 0.022 ms (CI 0.245–0.277). The fast path is therefore about twice as expensive *in compute*, which is the expected cost of two verifications; it sustains roughly 3,800 decisions/s on one core. Its benefit is elsewhere: it needs no transaction, no gas and no block, and it keeps working when the ledger is slow. The compute comparison is thus a cost, not a gain, and the real advantage depends on the network and consensus latency that we did not measure.

Table 6: Per-tier results: legitimate requests allowed and attacker requests accepted (%), threshold alone → full policy (θ min. trust, g anomaly gate)

Resource	θ	g	Legit.	Stealthy	Spoof
telemetry.read (T0)	0.50	1.00	100.0→100.0	100.0→100.0	100.0→100.0
historian.write (T1)	0.55	0.60	100.0→100.0	100.0→100.0	100.0→74.7
plc.setpoint.write (T2)	0.70	0.30	99.9→97.8	98.5→60.9	100.0→0.0
plc.firmware.update (T3)	0.78	0.20	96.0→94.8	52.2→13.6	52.4→0.0

7.5 Tiered Policy: Legitimate Acceptance and Attack Acceptance

Table 6 runs 4,000 legitimate requests (5% with a process upset) and 4,000 requests of each attack population through the four tiers, first with threshold alone and then with the full policy (gate and strict mode); eligibility is by device class, so the legitimate denominators shrink with the tier (4,000, 3,713, 1,145, 858). Fig. 2(a) shows the same data graphically.

Four observations follow. First, naive compromise is denied everywhere (0% accepted at all tiers), so it does not test the design. Second, at T0 an attacker that spoofs signals is accepted 100% of the time under any policy: this is by design, since T0 is read-only telemetry and we chose not to spend verification effort there, but it means T0 data can be read by a compromised device that keeps looking normal. Third, the gate and strict mode matter exactly where consequences are high: at T2 the full-spoof attacker falls from 100% to 0% accepted and the stealthy attacker from 98.5% to 60.9%; at T3 from 52.4% to 0% and from 52.2% to 13.6%. Fourth, this has a price. Legitimate allowance drops from 99.9% to 97.8% at T2 and from 96.0% to 94.8% at T3 (and 5.2% of legitimate T3 requests are denied outright), because upset requests exceed the gate and strict mode refuses borderline scores. A plant would decide whether those denials are acceptable for setpoint and firmware writes, where a second confirmation is usually welcome, and the next experiment shows the trade-off.

A short ablation of the trust signals, run in the base design, adds that the full score separates partial and stealthy mimicry with AUC 0.993 and 0.895 whereas dropping the behavioural signal leaves both at AUC 0.499–0.500, so the behavioural signal is the only one that separates mimicry from legitimate traffic in this setup; posture and network are drawn from the same ranges for both populations in the synthetic fleet, so removing them *raises* AUC, which argues for adaptive rather than fixed weighting and should not be read as evidence that real posture is uninformative.

7.6 Sensitivity to the Anomaly Gate

Fig. 2(b) sweeps the gate level from 0.15 to 0.60 for legitimate upset rates of 5% and 15% (0% gives no false denials by construction). Under a 5% upset rate, a gate of 0.30 blocks 40.1% of stealthy mimicry and 100% of full spoofing while denying 2.3% of legitimate requests. Tightening to 0.25 and 0.20 raises the stealthy block rate to 60.4% and 80.4% at 3.1% and 3.8% legitimate denials, and 0.15 blocks all of it at 4.6%. Loosening to 0.40 blocks no stealthy attacker and lets 25% of full spoofing through, rising to 74% at 0.60. At a 15% upset rate the same gates deny 7.4% (0.30) and 12.2% (0.20) of legitimate requests. The gate is thus a genuine trade-off whose operating point de-

Table 7: Detectability of attacks from TON_IoT telemetry (five seeds, 70/30 split, after de-duplication)

File	Rows	Att.%	Sup. AUC	IF AUC
Fridge	37,110	67	0.531	0.484
Modbus	17,792	78	0.623	0.498
Thermostat	32,350	55	0.513	0.497
Weather	39,260	62	0.972	0.819

pends on how noisy legitimate behaviour is, and because the attack range $[0.15, 0.40]$ is built to overlap legitimate upsets, no gate separates them completely; an attacker who keeps its anomaly below the gate is accepted, as Proposition 7(d) predicts.

7.7 Evaluation on Real Traffic: TON_IoT

The synthetic study fixes the anomaly distributions by hand, so we repeat the central experiments with the anomaly signal A produced by detectors on a public dataset. We use the TON_IoT collection, which provides labelled IIoT telemetry [40] and labelled network flows [41]; the heterogeneity of its features and attack types is analysed in [42]. The dataset licence grants free use for academic research on condition that the dataset papers are cited. We used four telemetry files (Fridge, Modbus, Thermostat, Weather) and the network file (211,043 flows, of which 190,474 remain after removing 20,569 exact duplicates; nine attack types plus normal traffic).

Method. Features exclude IP addresses and source ports, and for telemetry exclude dates and times, because normal and attack records were collected on different days. A gradient-boosted classifier (supervised) and an Isolation Forest trained on normal records only (unsupervised) are evaluated on a stratified 70/30 split repeated over five seeds. To approximate attacks unseen at training time, we also train the classifier with one attack type left out and score that type (seed 11). The classifier's attack probability is used as $A \in [0, 1]$. Legitimate requests carry the score of a normal test flow, attackers carry the score of an attack flow and spoof posture and network context to 1 (the full-spoof attacker of Section 6.2); posture, network context and the reliability warm-up stay synthetic, and because the files share no device identifier, assigning a flow-derived score to a device is arbitrary. The four tiers of Table 3 are used unchanged.

Telemetry alone is a weak signal. Table 7 shows that only the Weather file separates attacks from normal records usefully (supervised AUC 0.972, Isolation Forest 0.819); Modbus gives 0.623 and 0.498, and Thermostat and Fridge are close to chance (0.513 and 0.531 supervised). The Fridge result requires a warning: before whitespace stripping, one column (`temp_condition`) carries label-specific padding that gives a perfect AUC of 1.0, an encoding artefact that has nothing to do with device behaviour, so we strip all strings before training. We did not investigate why most telemetry is uninformative; one plausible reading is that the attacks target the services around a device rather than the physical readings it reports. The practical consequence is that a behavioural signal for a plant should come from traffic, not only from sensor values.

Network flows are a strong signal, with a caveat. On flows the supervised classifier reaches AUC 0.9999 (s.d. below 10^{-4}) and the Isolation Forest 0.916 ± 0.004, ranging from 0.813 (DDoS) to 0.956 (password attacks) by type. The near-perfect supervised value should be read as an upper bound: random splits of a benchmark whose attacks come from a few source hosts are optimistic, which is why we also test held-out types. With an attack type withheld, the AUC against normal traffic stays at 0.990–1.000, but the share of that type's flows scored above 0.5 falls to 65.4% for ransomware and 73.7% for man-in-the-middle, against 93–100% for most others. Legitimate flows are overwhelmingly scored near zero (median 0.0003), but the tail is heavy: the 99th percentile is 0.27 and the 99.9th is 0.96, so any gate cuts into some legitimate traffic (see the gate sweep below).

Table 8: Attacker requests accepted (%) with real TON_IoT anomaly scores: mean over nine attack types and worst single type; threshold alone (Thr.) versus full policy

Scenario	Tier	Thr. mean	Full mean	Thr. worst	Full worst
Seen types	T1	100.0	0.7	100.0	4.9
	T2	1.1	0.4	7.5	2.3
	T3	0.6	0.3	3.9	1.5
Held-out types	T1	100.0	11.9	100.0	41.6
	T2	15.9	7.9	51.5	33.9
	T3	9.3	6.6	36.7	35.8
Held-out, best 10%	T1	100.0	59.4	100.0	100.0
	T2	64.1	37.9	100.0	100.0
	T3	49.3	29.0	100.0	100.0
Held-out, best 1%	T1	100.0	76.5	100.0	100.0
	T2	87.4	74.7	100.0	100.0
	T3	76.0	58.2	100.0	100.0

Policy on real scores. Table 8 gives the percentage of attacker requests accepted, averaged over the nine attack types and, in the worst-case columns, for the single worst type, with threshold alone (Thr.) and with the full policy (gate and strict mode). Four findings follow. First, the full policy costs legitimate traffic little: allowance falls from 99.7, 99.5 and 99.0% to 99.6, 98.8 and 98.4% at T1, T2 and T3. Second, for attack types seen in training, the gate matters at T1 (mean acceptance 100% $\rightarrow$ 0.7%, worst 4.9%, man-in-the-middle), whereas at T2 and T3 the high thresholds already reject nearly all such flows (1.1% and 0.6% with threshold alone), because a real detector scores them near 1 and Proposition 7(d) then gives $T \approx 0.7$ or below. This corrects the synthetic study, in which the stealthy population was built to sit at $A \leq 0.4$ and therefore overstated how often the threshold alone fails on seen attacks. Third, for held-out types, residual acceptance grows to 11.9% (T1), 7.9% (T2) and 6.6% (T3) on average, and to 41.6%, 33.9% and 35.8% for the worst type (ransomware), so a policy that depends on a detector inherits the detector's blind spots. Fourth, an adaptive attacker that can observe the detector and send only its best-scoring flows is much more dangerous: if it can send only the lowest-scoring 10% of a held-out type's flows, mean acceptance at T1–T3 is 59.4, 37.9 and 29.0%, and 76.5, 74.7 and 58.2% for the best 1%, with some types always accepted. We assume here that the attacker can query the detector and craft traffic with a chosen score, which is a strong capability, so these numbers describe a worst case rather than a typical one.

Sweeping the gate on the real scores, a gate of 0.05 denies

3.6% of legitimate flows and blocks 100% of seen and 99.4% of held-out attack flows; 0.30 denies 1.0% and blocks 99.9% and 94.6%; 0.70 denies 0.3% and blocks 99.7% and 88.6%; and 0.90 denies 0.2% and blocks 99.5% and 82.9%. The real-score gate thus has a far better operating region than the synthetic one, because real attack scores are bimodal, but it degrades on unseen attack types. Detector hyper-parameters were left at library defaults, and the tier thresholds and gates are those fixed earlier for the synthetic study; none was tuned on these results.

7.8 A Second Dataset: WUSTL-IIOT-2018 (SCADA)

To check that the gate behaviour is not specific to one collection, we repeat the policy experiment on WUSTL-IIOT-2018, flow records captured on a SCADA testbed with injected attacks [43, 44]. The file has 7,037,983 rows and seven columns but only 62,959 distinct rows (99.1% duplicates; 39,817 normal and 23,142 attack), and it carries a binary label only. Two consequences follow. First, de-duplication is essential: a random split of the raw file puts identical records in training and test data. Second, there are no attack types, so the held-out-type test of Section 7.7 cannot be repeated. We exclude the source port, as before, which leaves five count features (total, source and destination packets, total and source bytes), and use a stratified 70/30 split on the unique rows over five seeds.

A gradient-boosted classifier is almost perfect (AUC 0.99998, sd 2×10^{-5}): attack flows in this testbed are short (a median of two packets, at most 22) and normal flows are not, so the task is nearly trivial and the result says little about hard attacks. An Isolation Forest trained on normal flows only reaches AUC 0.909 ± 0.006, a more realistic stand-in for an attack never seen in training. For the policy we map the Isolation Forest score to $A \in [0, 1]$ using normal training data alone (median to the 99.9th percentile), so no attack information enters the calibration.

Table 9 gives the results. With the supervised score the full policy accepts at most 0.3% of attack requests at T1–T3 and denies at most 0.2% of legitimate ones, essentially the TON_IoT picture. With the unsupervised score the gate and strict mode still reduce attack acceptance at the two critical tiers from 100% and 94.1% (threshold only) to 8.2% and 4.3%, but legitimate allowance drops to 87.5% at T2 and 82.2% at T3, a cost far above the 1–2% seen with the supervised detector, because many normal flows look unusual to a detector fitted on five coarse counters. At T0 and T1 the unsupervised variant accepts all attack requests: the T1 gate is 0.60, above the median attack score of about 0.49. For an adaptive attacker that sends only its best 10% (1%) of flows, acceptance with the unsupervised score is 86.9% (100%) at T2 and 43.0% (100%) at T3, against 2.1% (16.0%) and 0.7% (6.3%) with the supervised score. A gate sweep with the unsupervised score shows the same trade-off as before: a gate of 0.3 denies 12.5% of legitimate flows and blocks 91.3% of attack flows, and a gate of 0.5 blocks none. The finding that carries over is the role of the gate; the finding that does not is that a strong detector is available. When the detector is weak, as an unsupervised one on coarse features is, the policy trades a large share of legitimate availability for protection at the critical tiers. As with

Table 9: WUSTL-IIOT-2018: legitimate requests allowed and attack requests accepted (%) under the threshold-only and full policies, for two sources of the anomaly signal

	Supervised A			Isolation Forest A		
Tier	Legit	Att. thr.	Att. full	Legit	Att. thr.	Att. full
T0	100.0	100.0	100.0	99.3	100.0	100.0
T1	100.0	100.0	0.1	97.1	100.0	100.0
T2	100.0	0.3	0.3	87.5	100.0	8.2
T3	99.8	0.2	0.1	82.2	94.1	4.3

TON_IoT, posture and network context are synthetic and the flow-to-device pairing is arbitrary.

7.9 A Third Dataset With Attack Types: WUSTL-IIoT-2021

WUSTL-IIoT-2021 contains Argus flow records from an industrial testbed running Modbus traffic, with four labelled attack types [45, 46]. It lets us repeat the held-out-type test that the SCADA file of Section 7.8 could not support. The file has 1,194,464 rows, 1,107,448 of them normal and the rest DoS (78,305), reconnaissance (8,240), command injection (259) and backdoor (212). All records were captured on one day, attacks come from two to four source hosts, and one column (`IdleTime`) is an epoch timestamp that separates attack types by capture time. We therefore drop timestamps, IP addresses, the source port, `IdleTime` and the IP-identification fields, keeping 39 features, and then remove 41,645 exact duplicates before splitting (1,152,819 rows remain). The evaluation follows Section 7.7: a stratified 70/30 split over five seeds, a gradient-boosted classifier, an Isolation Forest trained on normal flows only (with A calibrated on normal training scores alone, as before), and leave-one-type-out models as the held-out proxy.

Detection. The supervised classifier is almost perfect on seen types (AUC 0.99999, and unchanged without the TTL fields, which fingerprint the attacker hosts). The Isolation Forest reaches AUC 0.987 ± 0.001, and between 0.983 and 0.990 for each attack type, because normal Modbus polling is very regular and the attacks (floods, scans) are far from it. Held-out detection is a different story: when DoS is removed from training, the classifier flags only 0.7% of DoS test flows at 0.5 (AUC against normal 0.71), and for command injection and reconnaissance the figure is 8.1% and 13.0% although their AUC is above 0.999, which means the ranking is right but the scores sit far below any useful gate. Backdoor is found at 66.7%. The command-injection and backdoor test sets contain only 74 and 63 flows, so those two figures are imprecise.

Policy. Table 10 gives the full-policy results as means over the four attack types. With seen types the full policy admits 3.6–4.2% of attack requests at T1–T3 (the average hides 10.8–12.1% for backdoor and 3.8–4.5% for command injection, the two rarest types) and denies none of the legitimate requests. With types withheld from training, acceptance rises to 75.4–77.8% on average and to 97.9–99.2% for DoS at the three higher tiers; the threshold-only policy is not much worse (last column), so on unseen types the supervised score contributes almost nothing, and an adaptive attacker sending its best 10% or 1% of flows is accepted in all cases. The unsupervised score

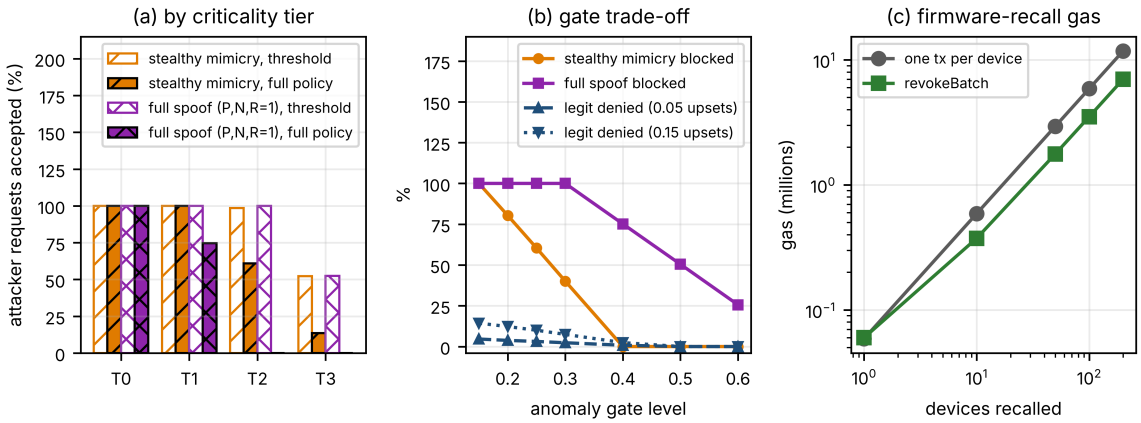


Figure 2: Industrial extensions: (a) attacker requests accepted per criticality tier under threshold-only and full policy; (b) anomaly-gate trade-off for 5% and 15% legitimate process-upset rates; (c) gas to recall devices with one transaction per device versus `revokeBatch`. Text description: in (a) the full policy accepts fewer attacker requests than the threshold-only policy for at least one attack type at each of T1–T3, and none of the full-spoof requests at T2–T3; in (b) the share of attacks blocked rises with the gate level while the share of legitimate requests denied stays below about 15%; in (c) recall by `revokeBatch` uses less gas than one transaction per device for every fleet larger than one device. Bars and lines are distinguished by hatching and markers as well as colour.

Table 10: WUSTL-IIoT-2021: legitimate requests allowed and attack requests accepted (%) under the full policy, as means over four attack types; "Thr.-only" is the threshold-only policy on the held-out supervised scores

	Supervised A			Isolation Forest A		
Tier	Legit	Seen	Held-out	Legit	Attack	Thr.-only[a]
T0	100.0	100.0	100.0	99.4	100.0	100.0
T1	100.0	3.6	77.8	98.5	0.0	100.0
T2	100.0	3.8	75.6	87.9	0.0	78.0
T3	100.0	4.2	75.4	77.7	0.0	77.8

behaves very differently on this data: the full policy admitted no attack request of any type at T1–T3, including the best 1% of an adaptive attacker's flows, at the price of denying 1.5%, 12.1% and 22.3% of legitimate requests at T1, T2 and T3. A gate of 0.5 on this score already blocks all attack flows while denying 2.3% of legitimate ones, whereas the T2 and T3 gates (0.30 and 0.20) deny 12.0% and 21.5%. The two datasets with attack types therefore agree on the main point and disagree on the detector: a supervised detector does not protect against attack types it has not seen, and a detector trained on normal traffic only can, but only when normal traffic is as regular as it is in this testbed.

Limits of this result. The favourable unsupervised numbers come from a laboratory testbed with machine-generated, nearly constant normal traffic and volumetric attacks from a handful of hosts, which makes anomaly detection easier than in a plant with operators, engineering workstations and maintenance windows. The rare attack types have very few flows, posture and network context are synthetic, and the pairing of a flow with a device is arbitrary.

7.10 Baseline Comparison: Revocation Exposure

On a shared logical clock we compare the per-request design with a session token (HMAC bearer token, TTL 60 or 300 s) and PKI with a CRL refreshed every 60 or 300 s, for 60 devices

Table 11: Revocation exposure (requests accepted after revocation; 60 devices, 1 req/s) and mean decision latency

Scheme	Mean exp.	Max exp.	Lat. (ms)
Session token (TTL 60 s)	32.5	59	0.005
Session token (TTL 300 s)	146.5	299	0.006
PKI+CRL (refresh 60 s)	33.5	60	0.074
PKI+CRL (refresh 300 s)	146.5	299	0.094
Proposed (per-request)	0.0	0	0.070

at 1 request/s with revocation times drawn from $U(100, 500)$ s. Table 11 shows that the audited design accepts no request after revocation, as Proposition 1 predicts, whereas the baselines accept on average 33 and 147 requests for the 60 s and 300 s windows (worst cases 59–299). The mechanism implies this result; the experiment quantifies it. Token validation is the cheapest per request (an HMAC, no public-key operation) and our design costs more per accepted request, which we do not hide. The fast path trades the zero for a bounded exposure of at most $\rho\Delta$ requests (Proposition 2): for example, with $\Delta = 5$ s and one request per second the bound is 5 requests, still far below the 33–147 of token and CRL schemes.

Three further baselines address the objection that tokens and CRLs are weak comparators (Table 12; 30 runs on the same revocation times, 60 devices, 1 request/s). Short-lived certificates with lifetime L and OCSP stapling with response validity V both bound exposure by the validity window, because a revoked device simply cannot renew; in our logical-clock model the two have identical exposure (29.0 requests for a 60 s window, 140.8 for 300 s) and differ only in who must be online (the CA for renewal, the responder for refresh), so we report them as the same mechanism and do not claim that one is better. A centralised online policy decision point that checks a revocation database on every request matches the proposed audited path with zero exposure, so exposure alone does not favour the blockchain design against this baseline. The differences are

Table 12: Additional baselines: requests accepted after revocation and availability of legitimate requests (30 runs, 60 devices, 1 req/s)

Scheme	Mean exp.	Max exp.	Avail. (%)
Short-lived cert (lifetime 60 s)	29.0	60	100.0
Short-lived cert (lifetime 300 s)	140.8	300	100.0
OCSP stapling (validity 60 s)	29.0	60	100.0
OCSP stapling (validity 300 s)	140.8	300	100.0
Central PDP (online, no outage)	0.0	0	100.0
Central PDP, 60 s outage, fail-closed	0.0	0	86.7
Central PDP, 60 s outage, fail-open	21.0	60	100.0

Table 13: Gas for recalling devices: one transaction per device versus `revokeBatch`, and `suspendBatch`

Devices	One tx each	`revokeBatch`	Saving	`suspendBatch`
1	58,862	59,885	-1.7%	31,626
10	588,620	373,739	36.5%	98,895
50	2,943,076	1,768,651	39.9%	397,927
100	5,885,960	3,512,303	40.3%	771,669
200	11,772,076	6,999,571	40.5%	1,519,249

elsewhere: the central PDP is a single operator who alone can revoke, reinstate or silently alter its log; if it is unreachable for 60 s it must either deny every request (86.7% of legitimate requests served) or accept requests from devices revoked in the meantime (21.0 on average, up to 60); and its decision log has no external anchor. Our fast path faces the same outage trade-off for revocations that happen while the replica is blind (Section 7.17), so the claim we can support is about governance and auditability, not availability. The simulation models validity windows only; real deployments add renewal failures, clock skew and responder caching that we did not model.

7.11 Firmware Recall Capacity

Table 13 reports gas for revoking k devices with one transaction each and with `revokeBatch` on the EVM; `suspendBatch` is shown for quarantine. For a single device the batch call is 1.7% more expensive because of the loop and array overhead, but from ten devices on it saves 36.5–40.5%, approaching about 35k gas per device against 58.9k. Taking a 30M-gas block, recalling 10^4 devices needs about 20 blocks with individual transactions and 12 with batches, and 5×10^4 devices need 99 and 59 blocks (derived from the measured per-device gas; other traffic and block interval are not modelled, so at a 2 s block time the larger recall would take on the order of two minutes at best). Revoking the highest-criticality devices first is therefore worth doing: devices revoked in later blocks stay exposed until their block.

7.12 Attack Detection

Four basic attacks were run in 500 independent trials each: replay, forged signature, use after revocation and probing with unregistered DIDs. All were denied in every trial. With 500 trials and no failure the one-sided 95% upper bound on the failure probability is $3/500 = 0.6\%$ (rule of three). This validates the state-machine logic against the implemented attacks; it is not evidence of unconditional security.

7.13 Decentralized Revocation Governance

Table 14 evaluates `RevocationGovernance` for $N \in \{5, 7, 9, 11, 15, 21\}$ with 200 trials per size. Mean quorum latency on the testbed grows roughly linearly, from 0.56 ms ($N = 5$) to 1.91 ms ($N = 21$) for a cooperative group and from 0.93 to 3.67 ms under maximal tolerated dissent ($N - q$ DENY voters), with 95% CI half-widths of at most 0.10 ms. Across all 1,200 trials the dissenting minority never prevented approval, and in 1,200 forced-revocation attempts with only $q - 1$ approvals no revocation executed, consistent with Proposition 6. On the EVM a proposal costs 72k gas, each vote 78k, and the final vote that triggers revocation 102k, so a quorum revocation costs 313k ($N = 5$) to 800k gas ($N = 21$): the governance cost grows with q, not with the fleet. Kumar et al. report 3–4 s for registration and under 5 s for revocation on a live PoA chain [28]; our figures are two to three orders of magnitude lower but measure compute only and are not comparable like for like. In a deployment each vote is a transaction, so revocation takes about q block confirmations when votes are sequential, or one to two blocks if voters submit in parallel.

Table 14: Governance: mean testbed latency (ms) cooperative and under maximal dissent; % trials where maximal dissent still reached quorum (Appr.); % where a $q - 1$ coalition executed a revocation (Forced); EVM gas for one quorum revocation

N	q	Coop.	Dissent	Appr.%	Forced%	Total gas
5	3	0.562	0.926	100	0	313,402
7	4	0.667	1.166	100	0	374,184
9	5	0.803	1.440	100	0	435,062
11	6	0.946	1.729	100	0	495,892
15	8	1.299	2.438	100	0	617,552
21	11	1.909	3.665	100	0	800,030

7.14 End-to-End Evaluation on a Four-Validator QBFT Network

To move beyond in-process execution we deployed the three contracts on a private Hyperledger Besu 25.7.0 network of four validators running QBFT with a 2 s block period, a 4 s round-change timeout, chain ID 2026 and zero base fee, and drove it through JSON-RPC with signed transactions (`besu_benchmark.js`, `besu_fault.js`). In this first deployment all four validators and the load generator ran on the same two-vCPU container, so the network is real in the sense that blocks are proposed, voted on and committed by separate processes, but there is no wide-area delay and the processes compete for the same two cores. The numbers below therefore describe consensus-bound behaviour on a small host; Section 7.15 repeats them across regions.

Latency and finality. Table 15 shows that every state-changing call needs about one block period from submission to receipt (1.98–2.00 s on average, p95 about 2.2 s), independent of the operation; QBFT finalizes in the block that includes the transaction, so a receipt is also a final result. Revocation became visible when queried on a second validator after 1.87 s on average, and 30 of 30 audited `evaluateAccess` calls issued after that point returned DENY. The measured difference between the submitting node's receipt and the replica's view was -117 ± 75 ms (the replica was seen to be up to date before the receipt poll returned, the maximum being +27 ms), which is within our polling jitter of a few milliseconds to tens of milliseconds. We therefore read the replica lag as $\Delta \approx 0$ relative to the block period on a single host and, for the exposure bound $\rho\Delta$ of Proposition 2, take the block period as the dominant term

Table 15: Four-validator QBFT network (Besu 25.7.0, 2 s blocks, single host): latency, finality and quorum revocation

Measurement	Mean ± sd	p95	n
Block interval (s)	2.00 ± 0	2.00	40
`register` submit→receipt (ms)	1977 ± 139	2165	40
`evaluateAccess` submit→receipt (ms)	1997 ± 125	2179	40
`anchorDecisions` submit→receipt (ms)	1993 ± 127	2185	20
Revocation submit→receipt (ms)	1985 ± 121	2153	30
Revocation submit→visible on 2nd validator (ms)	1868 ± 85	2018	30
Quorum revocation, N=5, q=3, sequential votes (ms)	8067 ± 237	8680	10
Quorum revocation, N=5, q=3, parallel votes (ms)	4033 ± 116	4156	10

Table 16: Throughput of audited decisions on the same network (10 senders, pre-signed)

Offered decisions	Mined	Throughput (tx/s)	Mean latency (s)
100	100	17.8	5.4
300	300	16.0	14.7
600	600	11.3	31.6 (p95 50.8)

for the time between a revocation request and its enforcement.

Throughput. Table 16 reports audited decisions sent as pre-signed transactions from ten sender accounts. All transactions were mined at the three offered loads, but sustained throughput was 11–18 tx/s and latency grew with the queue (mean 31.6 s at 600 offered). This is 12–19 times below the gas-based ceiling of about 213 decisions per second implied by 426 decisions per 30M-gas block at 2 s, and the ceiling quoted in Section 7.3 should be read as an upper bound, not an estimate. We attribute the gap to CPU saturation (four JVMs and the load generator on two cores, with signature recovery and EVM execution for every transaction), but we did not profile it; the multi-region run of Section 7.15 gave higher throughput, which is consistent with this explanation without proving it. The practical consequence for the design is unchanged and strengthened: routine requests must not be sent through consensus.

Governance and recall. With five voters and quorum three, a revocation completed in 8.07 s when votes were submitted one after another (each waiting for its block, so about four blocks) and in 4.03 s when the votes were submitted together (about two blocks); all 20 proposals ended in revocation. A batch of 50 devices was revoked in one transaction (1.77M gas) and confirmed after 2.95 s, and a batch of 200 devices (7.00M gas) after 6.13 s; sampled revocations were visible on another validator. By comparison, registering 200 devices one transaction at a time took 399 s. The batch and parallel-vote results confirm that on a block-timed chain, the number of blocks rather than the number of transactions determines recall and governance latency.

Fault tolerance. With four validators QBFT tolerates one fault. With one validator stopped, registration latency had a mean of 2.83 s (sd 2.00 s, p95 6.16 s, $n = 8$) against 1.97 s at baseline: the median was unchanged (1.91 s), but rounds in which the stopped validator was the proposer had to time out and change round. With two validators stopped no block was produced and a pending transaction was not mined within 20 s; after both were restarted it was mined after 78 s. These runs used a single pending transaction and small samples, so they show the qualitative behaviour (safe halt below the quorum, resumption after recovery) rather than a tight latency estimate. For the architecture, they mean that the fast path keeps deciding on its last replicated state during a halt and the outage policy of Section 4.8 governs what is allowed meanwhile.

7.15 Four Validators in Four Regions

To remove the shared-CPU limitation and add real network distance, we repeated the same benchmark on Google Cloud with one validator per e2-standard-2 virtual machine (2 vCPU, 8 GB) in europe-west3, us-east4, asia-southeast1 and southamerica-east1 (same Besu 25.7.0 and QBFT parameters as above; Ubuntu 24.04 and OpenJDK 21). The load generator ran in europe-west3 and submitted through the European validator's endpoint; the revocation-visibility test queried two validators in other regions (endpoints 3 and 4 in the run configuration, which are the Asian and South American ones). We injected no artificial delay and did not record round-trip times, so the delay is whatever the provider's inter-region network gave on the day of the run (2026-10-12). Table 17 sets the results next to the single-host run. Both are single runs of the same script, with the sample sizes of Table 15.

Finality and latency. Block production stayed at 2.00 s with no deviation, and the latency of `register`, `evaluateAccess` and `anchorDecisions` was unchanged within noise (1.98–2.00 s, p95 2.16–2.18 s). Consistent with this, inter-region distance did not push consensus beyond one block period on this network; the 2 s timer, not the network, set finality. The revocation receipt took 1.55 s on average, less than on one host; we did not investigate this, the submissions are not aligned to the block timer, and we do not read it as a speed-up.

Replica lag. A revocation became visible on a validator in another region 1.76 s after submission, and 30 of 30 audited decisions on a second remote validator issued afterwards returned DENY. Unlike on one host, the replica now trailed the receipt: by $+212 \pm 145$ ms on average (p95 362 ms, maximum 495 ms, minimum -39 ms). Taking the largest value as an estimate for this network, the time between a revocation request and its visibility to a remote replica was about one block period plus 0.2–0.5 s, so the exposure bound $\rho\Delta$ of Proposition 2 for a fast-path gateway is dominated by the block period but no longer negligible against the network. This is one run on one provider's backbone; a plant link over a public network or a satellite path would give a larger Δ and has to be measured where it is deployed.

Throughput. All 100, 300 and 600 pre-signed audited decisions were mined, at 30.8, 34.4 and 50.1 tx/s, against 17.8, 16.0 and 11.3 on one host; the mean latency at 600 offered fell from 31.6 s to 8.6 s (p95 10.7 s). This is consistent with the CPU contention suspected earlier, but throughput still rose with offered load, so we did not find the saturation point, and a figure of 50 tx/s is a lower bound for this deployment, not its capacity. It remains 4–7 times below the gas-based ceiling of 213 decisions per second. We did not determine what limits it (block gas, round timing or the load generator), and the conclusion for the design is unchanged: routine requests must not wait for consensus.

Governance and recall. A three-of-five revocation took

Table 17: The same benchmark on one host and with one validator in each of four Google Cloud regions (mean ± sd; single runs)

Measurement	One host	Four regions
Block interval (s)	2 ± 0	2 ± 0
`register`, submit→receipt (ms)	1977 ± 139	1980 ± 162
`evaluateAccess`, submit→receipt (ms)	1997 ± 125	1997 ± 134
`anchorDecisions`, submit→receipt (ms)	1993 ± 127	1993 ± 130
Revocation, submit→receipt (ms)	1985 ± 121	1552 ± 169
Revocation, submit→visible on replica (ms)	1868 ± 85	1764 ± 211
Replica minus receipt (ms)	-117 ± 75	212 ± 145
Quorum revocation N=5, q=3, sequential (s)	8.07 ± 0.24	8.17 ± 0.14
Quorum revocation N=5, q=3, parallel (s)	4.03 ± 0.12	4.33 ± 0.13
`revokeBatch`, 50 devices, receipt (s)	2.95	2.41
`revokeBatch`, 200 devices, receipt (s)	6.13	2.99
Throughput, 100 offered (tx/s)	17.8	30.8
Throughput, 300 offered (tx/s)	16.0	34.4
Throughput, 600 offered (tx/s)	11.3	50.1
Mean latency at 600 offered (s)	31.6	8.6

8.17 s with sequential votes and 4.33 s with parallel votes (all 10 trials each ended in revocation), the same four and two blocks as on one host (8.07 and 4.03 s). Revoking 50 and 200 devices in one transaction was confirmed after 2.41 and 2.99 s for 1.77M and 7.00M gas, the same gas as before, and the sampled revocations were visible on another validator; the single-host run needed 2.95 and 6.13 s. Registering the devices one by one still took 99.7 s for 50 and 399 s for 200, about one block each, so the argument for batching in Section 4.6 is unchanged.

Fault tolerance. With one validator stopped, registration took 3.43 ± 2.17 s (p50 2.14 s, p95 6.29 s, $n = 8$) against 2.83 s on one host; the median stayed near the baseline and the mean reflects the rounds in which the stopped validator was the proposer. With two validators stopped no block was produced and a pending transaction was not mined within 20 s; after the validators were restarted it was mined after 75.0 s (78 s on one host). As before, these runs used one pending transaction and few samples and show the qualitative behaviour. We did not test network partitions, injected delay or validators in the same region.

7.16 Static Analysis and Randomised Invariant Tests

We ran Slither on the three contracts with its 102 detectors. It reported 11 findings: one medium (a strict equality on `registeredAt==0` in `getStatus`, which is the intended existence test), four low (timestamp-comparison warnings on the same existence test and on a status check, a missing zero-address check on the oracle in the constructor, and an event emitted after the external registry call in `castVote`, whose callee is our own trusted registry), one informational finding per signature-recovery routine (inline assembly) and four optimisation hints (state variables that could be immutable). No reentrancy, access-control or arithmetic finding was reported. Reading the code for what Slither does not check, we found one genuine gap: if the oracle address were zero, or a device were registered with a zero signer, an invalid signature (for which `ecrecover` returns the zero address) would pass. We added a zero-oracle check in the constructor and rejected a zero recovered address at registration; the measured cost is +62 gas for `registerDevice` and none for the other operations. We did not apply the immutable hints, to keep the measured gas comparable across the evaluation. Signature malleability is harmless here because replay protection is keyed on (device, nonce), not on the signature bytes.

Table 18: Outage scenario (policy-defined, 400 devices, 1 req/s): availability of legitimate requests and unsafe accepts

Outage (s)	Revoked devices	Avail. (%) closed	Avail. (%) degr.	Unsafe accepts	T2–T3 denied
10	43	98.3	99.9	155	100%
30	40	94.9	99.7	456	100%
60	36	89.9	99.4	962	100%
120	44	79.7	99.0	2,149	100%
300	30	50.4	97.0	4,398	100%

Three new tests complement the 13 unit tests: the zero-oracle and zero-signer cases, and a randomised state-machine test that applies 300 steps (three seeds, six devices) of suspend, reinstate, revoke and access operations against an independent model. It checks that revocation is terminal, that reinstatement succeeds only from the suspended state, that an inactive device or a reused (device, nonce) pair is never allowed, and that the on-chain status always equals the model status. All 16 tests pass. This is bounded random testing, not a proof and not an audit; the contracts have not been formally verified.

7.17 Degraded Operation During an Outage

Table 18 applies the policy of Section 4.8 to a scenario with 400 devices, one request per second each, a quarter of the devices acting as T2 clients that issue a T2 request every fourth second, and 10% of the devices revoked at random times during an outage that starts at $t = 200$ s of 600 s. A fully fail-closed PEP stops serving everyone for the whole outage, so availability falls to 89.9% for a 60 s outage and 50.4% for 300 s. The degraded policy keeps T0–T1 running (99.4% and 97.0%) but denies every T2 request during the outage and accepts requests from devices revoked while it was blind (962 requests for 60 s, 4,398 for 300 s). This scenario follows directly from the stated policy and is deterministic given its parameters, so it is an illustration of the trade-off, not an empirical result; its use is to show that the operator chooses between unsafe acceptances and lost availability, and that the choice is explicit.

8 Discussion

8.1 What the Design Buys and What It Costs

Compared with tokens and CRLs, the central gain is the removal of an implicit-trust window: on the audited path a revoked device loses access at the next call, and on the fast path within the replica lag, a bound the operator can set and measure (Propositions 1 and 2; 33–147 accepted requests for the baselines against none or $\rho\Delta$). Quorum governance removes the single-registrar insider (T7) at a gas cost that scales with the voter group rather than the fleet. The tiered policy concentrates verification effort where a wrong decision hurts: at T2–T3 the gate and strict mode cut a spoofing attacker's acceptance from about 100% to 0% and from about half to zero, and the cost is a few percent of legitimate requests denied when the process is upset. In contrast to schemes that authenticate once and trust the session [20], nothing here is trusted by default; in contrast to conceptual smart-factory designs [18], the policy is an executable contract with measured cost.

The costs are equally concrete. The fast path doubles per-request compute (two signature checks), the audited path is

bounded by block gas (about 426 decisions per 30M-gas block by gas, and 11–18 per second measured on a single host and 31–50 per second with validators in four regions), recall of very large fleets takes tens of blocks, and the whole design adds a consortium chain, a replica and an oracle to operate. For a plant that has one owner and a good internal PKI, this is probably unjustified; the argument for it is multi-party plants in which revocation authority and audit trails must be shared.

8.2 Deployment Guidance

Tier assignment. Tiers should follow consequence, not protocol: any action whose misuse could damage equipment or product, or alter safety-relevant logic, belongs on the audited, strict path; high-rate reads belong on the fast path. *Gate calibration.* The anomaly gate should be set from a plant's own distribution of legitimate deviations, including start-up and batch changes; Section 7.6 shows the operating point moves substantially with the upset rate. *Replica lag.* Δ should be measured, monitored and alarmed, since Proposition 2 is only as good as A5. *Recall rehearsal.* Batch revocation should be rehearsed with quarantine (`suspendBatch`) first, because REVOKED is terminal and a mistaken revocation requires re-enrolment with a new key. *Gas economics.* On a private PoA network gas can be zero-priced, so the figures above indicate capacity and relative cost rather than money; gasless relaying can hide fees from constrained devices [38].

8.3 Threats to Validity

Internal validity. In the main experiments, devices, signal ranges and attacks are synthetic and chosen by us, and the legitimate and attacking anomaly ranges overlap by construction. In the TON_IoT study only the anomaly signal is real: posture, network context and reliability history are synthetic, the assignment of a flow score to a device is arbitrary, attackers are assumed to spoof posture and network context perfectly, and the held-out-type test approximates but is not a real zero-day setting. The dataset itself has known weaknesses that we saw directly: exact duplicates, collection of normal and attack records on different days, an encoding artefact in one file, and attacks from a few source hosts, so the supervised AUC of 0.9999 is optimistic. None of this is a plant measurement. The baselines are minimal logical-clock models: short-lived certificates and OCSP stapling are modelled only by their validity window, and a production PKI would add renewal failures, clock skew and caching. The outage result is policy-defined. *External validity.* The simulations ran on one two-vCPU container with the two tracks using different signature schemes; no measurement was made on a microcontroller or on industrial protocols. The first Besu network ran on that same host over the loopback interface, so its throughput is limited by CPU contention; the second used four cloud regions on one provider's backbone, as a single run on one day, without injected delay or partitions, and the fault tests used a single pending transaction and small samples. *Construct validity.* In the simulation, revocation exposure is counted on a logical clock and ignores propagation and finality time; the Besu measurements give the finality component (one block period) and, for four regions, a replication lag of about 0.2 s, but not the delay of an industrial network. *Conclusion validity.* Intervals come from 10–30 runs on one host or one four-region deployment; the 500-trial attack tests have a 0.6% rule-of-three bound.

8.4 Limitations and Future Work

First, the multi-node measurements come from one single-host run and one four-region run on a single cloud provider, without injected delay, partitions or a plant network; the replica lag $\Delta \approx 0.2\,\text{s}$, the throughput lower bound of 50 tx/s and the failure behaviour have to be re-measured on the intended deployment. Second, the PEP is trusted for request-signature checking in Track A, and a single oracle and registrar remain single points of compromise (A4); a threshold-signed oracle [32] and a quorum registrar set would remove them. Third, our legacy-device proxy is a design with a modelled class, not an implemented gateway, and no real fieldbus or OPC-style protocol was exercised. Fourth, trust scores and anomaly values appear in transaction data and are visible to consortium members; differential-privacy protection of exposed scores [39] is not implemented. Fifth, adversarial-AI mimicry is richer than our populations [29], and the signal producers (attestation, process-aware anomaly detection, possibly cross-site learning [30]) are inputs we assume. Sixth, multi-PEP zones require binding the PEP identity into requests (Proposition 3). Seventh, the policy parameters were hand-tuned, and learning them from plant data [15] is open. Eighth, the real-data study shows that a detector-dependent gate inherits the detector's blind spots on unseen attacks and is exposed to an adaptive attacker who can steer its own score; adversarially robust detection, rate limiting of low-score probing and joint use of several detectors are the natural defences, and we did not evaluate them. Ninth, the SCADA dataset has only a binary label and 99.1% duplicate rows, and the dataset with attack types is a laboratory testbed with very regular normal traffic and attacks from a few hosts; neither is a plant, and two of its attack types have fewer than 300 flows. Tenth, a field study is missing: per-device telemetry joined to the traffic of the same device, with real attestation, would replace the arbitrary pairing used here. Finally, the contracts are research code: they pass 16 tests, including a randomised state-machine test, and a Slither run found no serious issue after one hardening fix, but they have not been audited or formally verified, and the literature on smart-contract quality assurance [33] should be followed before any production use.

9 Conclusion

This paper proposed a blockchain-enabled Zero Trust architecture for identity management in Industrial IoT, built around the observation that a plant needs multi-party revocation authority, verification proportional to consequence, and an enforcement path that does not wait for consensus. Three smart contracts anchor identity, tiered policy and quorum-governed revocation; a fast path decides locally against replicated state and anchors Merkle roots of its decisions, while resources whose misuse is costly are decided on-chain with an anomaly gate and a strict mode; and batch revocation supports firmware recalls. We proved immediate revocation on the audited path, a $\rho\Delta$ exposure bound on the fast path, replay resistance, attestation unforgeability, key-reuse prevention and governance

safety and liveness, and showed exactly when a spoofing attacker is accepted.

On a synthetic industrial fleet, the audited design accepted no request after revocation where token and CRL baselines accepted 33–147 on average; the gate and strict mode reduced full-spoof acceptance at the two most critical tiers to 0% and stealthy-mimicry acceptance to 60.9% and 13.6%, at a cost of 2.2–5.2% denied legitimate requests; batch revocation saved about 40% of gas, with about 35k gas per device; anchoring cost 54 gas per decision at 1,000 decisions per root; and quorum revocation for 5–21 voters cost 313k–800k gas, with no coalition below quorum succeeding in 1,200 attempts. With the behavioural signal taken from real TON_IoT traffic instead, the full policy accepted under 1% of seen-type attacker requests at the critical tiers and 6.6–11.9% of held-out-type requests on average (worst type 33.9–41.6% at T1–T2), while an adaptive attacker that sends only its best-scoring flows was accepted 29–77% of the time; telemetry alone was a weak signal for most devices. Two further industrial datasets (WUSTL-IIOT-2018 and WUSTL-IIoT-2021) agree on the role of the gate and disagree on the detector: a supervised detector was accepted on 75–78% of attack requests of withheld types on average on the one with attack types, whereas an unsupervised detector trained on normal traffic blocked them all at the upper tiers on that laboratory testbed, denying 12–22% of legitimate requests at the two critical tiers. A centralised online policy decision point matches the zero exposure of the audited path, so the case for the design rests on shared revocation authority and an externally anchored audit trail, not on exposure alone. On a four-validator Besu QBFT network, audited calls were final in about one 2 s block, a revocation was visible on a second validator in 1.9 s and denied all 30 later audited requests, a three-of-five quorum revocation took 4–8 s, sustained throughput was 11–18 decisions per second on one two-vCPU host and at least 31–50 with the validators in four cloud regions, where a revocation reached a remote validator about 0.2 s after the submitting node's receipt, and the chain halted safely when two of four validators were stopped. The remaining timing numbers are compute-bound, the posture and network signals remain synthetic, and an attacker that keeps its behaviour within normal variation is accepted at every tier. Whether the approach holds up in a real plant is a question for deployment on a plant network with real protocols, devices and telemetry, which we regard as the next step together with threshold-signed oracles and privacy protection for exposed scores. Code, result files and generation scripts are released at github.com/irfanrazanaqvi/BCZT_IDM_IoT_experments_code; the TON_IoT data must be obtained from its owners.

References

[1] K. Stouffer *et al.*, "Guide to Operational Technology (OT) Security," NIST Special Publication 800-82 Rev. 3, National Institute of Standards and Technology, Sep. 2023. [Online]

[2] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, National Institute of Standards and Technology, 2020. [Online]

[3] E. B. Fernandez and A. Brazhuk, "A critical analysis of zero trust architecture (ZTA)," *Computer Standards & Interfaces*, vol. 89, Art. no. 103832, 2024. [Online]

[4] P. Phiayura and S. Teerakanok, "A comprehensive framework for migrating to zero trust architecture," *IEEE Access*, vol. 11, pp. 19487–19511, 2023. [Online]

[5] M. W. Song, T. Nguyen, and C. Irvine, "A zero trust architecture for critical operational technology systems," in *Proc. 58th Hawaii Int. Conf. Syst. Sci. (HICSS)*, 2025. [Online]

[6] C. Zanasi, S. Russo, and M. Colajanni, "Flexible zero trust architecture for the cybersecurity of industrial IoT infrastructures," *Ad Hoc Networks*, vol. 156, Art. no. 103414, 2024. [Online]

[7] M. Sporny *et al.*, "Decentralized Identifiers (DIDs) v1.0," W3C Recommendation, Jul. 2022. [Online]

[8] S. M. Hosseini, J. Ferreira, and P. C. Bartolomeu, "Blockchain-based decentralized identification in IoT: an overview of existing frameworks and their limitations," *Electronics*, vol. 12, no. 6, p. 1283, 2023. [Online]

[9] A. D. Aguru and S. B. Erukala, "A blockchain-based self-sovereign identity management framework for Internet of Things," in *Recent Trends in Intelligent Systems and Next Generation Wireless Communication (IIWCS 2024)*, Lecture Notes in Networks and Systems, vol. 1329. Singapore: Springer, 2025, pp. 299–309. [Online]

[10] G. Folino, A. Forestiero, and G. Papuzzo, "Self-sovereign identification of IoT devices by using physically unclonable functions and blockchain," in *Numerical Computations: Theory and Algorithms (NUMTA 2023)*, Lecture Notes in Computer Science, vol. 14478. Cham: Springer, 2025. [Online]

[11] Z. Zhai *et al.*, "A lightweight authentication method for Industrial Internet of Things based on blockchain and Chebyshev chaotic maps," *Future Internet*, vol. 17, no. 8, Art. no. 338, 2025. [Online]

[12] M. Essaid and H. Ju, "Blockchain solutions for enhancing security and privacy in Industrial IoT," *Applied Sciences*, vol. 15, no. 12, Art. no. 6835, 2025. [Online]

[13] C. Liu, R. Tan, Y. Wu, Y. Feng, Z. Jin, F. Zhang, Y. Liu, and Q. Liu, "Dissecting zero trust: research landscape and its implementation in IoT," *Cybersecurity*, vol. 7, no. 1, Art. no. 20, 2024. [Online]

[14] S. Nie, J. Ren, R. Wu, P. Han, Z. Han, and W. Wan, "Zero-trust access control mechanism based on blockchain and inner-product encryption in the Internet of Things in a 6G environment," *Sensors*, vol. 25, no. 2, p. 550, 2025. [Online]

[15] M. M. Salim, M. Kim, S. K. Singh, and J. H. Park, "Zero-trust blockchain-enabled framework for scalable and secure IoT networks (ZT-BlocIoT)," *Future Generation Computer Systems*, vol. 175, Art. no. 108093, 2026. [Online]

[16] R. Mukta, S. Pal, K. Chowdhury, M. Hitchens, H. Paik, and S. S. Kanhere, "Zero trust-driven access control delegation using blockchain," *Blockchain: Research and Applications*, vol. 7, no. 1, Art. no. 100319, 2026. [Online]

[17] E. Jamwal, P. Srivastava, and P. S. Chakraborty, "Blockchain-enabled zero-trust framework for secure cloud data sharing: survey," in *Proc. 2026 Int. Conf. Intell. Syst. Eng., Secured Syst. Cybersecurity (ICISESSC)*, 2026. [Online]

[18] M. Farhan *et al.*, "ZTF-SF: a zero-trust blockchain-enabled IIoT framework for secure smart factories," in *Proc. 2026 2nd Int. Conf. Comput. Intell. Approaches Appl. (ICCIAA)*, 2026. [Online]

[19] K. Potturi, B. Gautam, C. S. Pareek, A. Gupta, S. C. Chava, and B. S. Katta, "A hybrid AI and blockchain-enabled zero-trust security framework for cloud, edge, and IoT systems," in *Proc. 2026 Int. Conf. Comput. Theory Wireless Commun. (ICCTWC)*, 2026. [Online]

[20] Z. Wang, Y. Guo, and Y. Guo, "Blockchain-based zero-trust continuous authentication scheme for medical IoT," *Journal of Parallel and Distributed Computing*, vol. 217, Art. no. 105333, 2026. [Online]

[21] M. James, T. Newe, D. O'Shea, and G. D. O'Mahony, "Authentication and authorization in zero trust IoT: A survey," in *Proc. Irish Signals and Systems Conf. (ISSC)*, 2024. [Online]

[22] R. Kumar and R. Sharma, "A comprehensive approach to industry 4.0 security: blockchain and dynamic access control integration," in *Proc. 2024 6th Int. Conf. Computational Intelligence and Communication Technologies (CCICT)*, 2024, pp. 477–484. [Online]

[23] M. A. Azad, S. Abdullah, J. Arshad, H. Lallie, and Y. H. Ahmed, "Verify and trust: a multidimensional survey of zero-trust security in the age of IoT," *Internet of Things*, vol. 27, Art. no. 101227, 2024. [Online]

[24] N. Nahar, K. Andersson, O. Schelén, and S. Saguna, "A survey on zero trust architecture: applications and challenges of 6G networks," *IEEE Access*, vol. 12, pp. 94753–94764, 2024. [Online]

[25] J. Ren, J. Zhang, Y. Ren, and J. Xu, "Blockchain-based self-sovereign identity management mechanism in AIoT environments," *Electronics*, vol. 14, no. 19, p. 3954, 2025. [Online]

[26] A. Philipp and A. Küpper, "DAXiot: A decentralized authentication and authorization scheme for dynamic IoT networks," arXiv:2307.06919, 2023. [Online]

[27] C. Mazzocca, A. Acar, S. Uluagac, and R. Montanari, "EVOKE: Efficient revocation of verifiable credentials in IoT networks," in *Proc. 33rd USENIX Security Symp.*, Philadelphia, PA, USA, Aug. 2024, pp. 1279–1295. [Online]

[28] A. Kumar, B. Sharma, R. K. Kaushal, and I. Ben Dhaou, "Privacy-preserving

identity management system leveraging blockchain technology," in *Proc. 15th IEEE Int. Conf. Commun. Syst. Netw. Technol. (CSNT)*, 2026, p. 625 ff. [Online]

[29] M. Usama, A. Aziz, N. Alasbali, N. Alturki, M. Hanif, and M. Ur Rehman, "Blockchain-enabled identity management for IoT: a multi-layered defense against adversarial AI," *Scientific Reports*, vol. 16, Art. no. 4371, 2026. [Online]

[30] A. K. Tom, A. Khraisat, T. Jan, M. Whaiduzzaman, T. D. Nguyen, and A. Alazab, "Survey of federated learning for cyber threat intelligence in Industrial IoT: Techniques, applications and deployment models," *Future Internet*, vol. 17, no. 9, Art. no. 409, 2025. [Online]

[31] W. Shujaa, M. Alanzi, and S. Sankaranarayanan, "Enhancing IoT security through blockchain integration," *Frontiers in Computer Science*, vol. 7, Art. no. 1670473, 2025. [Online]

[32] A. Hassan, I. Makhdoom, W. Iqbal, A. Ahmad, and A. Raza, "From trust to truth: Advancements in mitigating the blockchain oracle problem," *J. Netw. Comput. Appl.*, vol. 217, Art. no. 103672, 2023. [Online]

[33] Z. Wei *et al.*, "Survey on quality assurance of smart contracts," *ACM Comput. Surv.*, vol. 57, no. 2, Art. no. 32, 2024. [Online]

[34] M. T. Ta and T. Q. Do, "A study on gas cost of Ethereum smart contracts and performance of blockchain on simulation tool," *Peer-to-Peer Networking and Applications*, vol. 17, no. 1, pp. 200–212, 2024. [Online]

[35] M. R. Hasan, A. Alazab, S. B. Joy, M. N. Uddin, M. A. Uddin, A. Khraisat, I. Gondal, W. F. Urmi, and M. A. Talukder, "Smart contract-based access control framework for Internet of Things devices," *Computers*, vol. 12, no. 11, Art. no. 240, 2023. [Online]

[36] E. U. Haque, A. Shah, J. Iqbal, S. S. Ullah, R. Alroobaea, and S. Hussain, "A scalable blockchain based framework for efficient IoT data management using lightweight consensus," *Scientific Reports*, vol. 14, Art. no. 7841, 2024. [Online]

[37] A. K. Peepliwal, H. M. Pandey, S. Prakash, A. A. Mahajan, S. S. Chowhan, V. Kumar, and R. Sharma, "A prototype model of zero trust architecture blockchain with EigenTrust-based practical Byzantine fault tolerance protocol to manage decentralized clinical trials," *Digital Communications and Networks*, 2024. [Online]

[38] Y. Madhwal *et al.*, "Empowering autonomous IoT devices in blockchain through gasless transactions," *Blockchain: Research and Applications*, vol. 6, no. 2, Art. no. 100257, 2025. [Online]

[39] L. Ren, H. Li, Y. Liu, D. Liao, M. Zhang, and H. Jin, "Identity trust management based on differential privacy for internet of vehicles," *Scientific Reports*, vol. 16, Art. no. 20921, 2026. [Online]

[40] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems," *IEEE Access*, vol. 8, pp. 165130–165150, 2020. [Online]

[41] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets," *Sustainable Cities and Society*, vol. 72, Art. no. 102994, 2021. [Online]

[42] T. M. Booij, I. Chiscop, E. Meeuwissen, N. Moustafa, and F. T. H. den Hartog, "ToN_IoT: The role of heterogeneity and the need for standardization of features and attack types in IoT network intrusion data sets," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 485–496, 2022. [Online]

[43] M. A. Teixeira, T. Salman, M. Zolanvari, R. Jain, N. Meskin, and M. Samaka, "SCADA system testbed for cybersecurity research using machine learning approach," *Future Internet*, vol. 10, no. 8, Art. no. 76, 2018. [Online]

[44] M. A. Teixeira, M. Zolanvari, and R. Jain, "WUSTL-IIOT-2018 dataset for ICS (SCADA) cybersecurity research," IEEE DataPort, 2018. [Online]

[45] M. Zolanvari, M. A. Teixeira, L. Gupta, K. M. Khan, and R. Jain, "Machine learning-based network vulnerability analysis of industrial Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 4, pp. 6822–6834, 2019. [Online]

[46] M. Zolanvari, "WUSTL-IIOT-2021," IEEE DataPort, 2021. [Online]